

天津工业大学

软件学院 · 实习报告

基于Vue和Node的todolist设计与开发

课程：	安全程序设计实习
班级：	网安2301
学号：	2311320111+2311320105
姓名：	吕仔贤+丁少亭
成绩：	

一、项目背景与目标

1.1 项目背景

随着互联网技术的飞速发展，Web 应用已成为日常生活中不可或缺的一部分。用户对于应用性能、响应速度以及数据安全性的要求日益严苛。传统的客户端、服务端分离的开发模式中，往往暴露出协作困难、产品交付速度慢、RESTful API 端点难以维护等等问题。而传统 SPA 采取的客户端渲染方式也有对低性能客户端不友好、首屏闪烁严重、SEO 不佳等等“顽疾”。为解决这些挑战，并提供更优质的用户体验，现代 Web 开发正积极探索并采纳新的技术栈和架构模式。

本项目致力于设计与开发一个现代化的 To-Do List 应用，旨在提供高效的任务管理能力。为实现这一目标，我们选择了以下先进且高度集成的技术栈：

- 前端SSR框架：Vue + Vike。结合Vue的组件化开发优势与 Vike 提供的开箱即用型服务器端渲染（SSR）能力，旨在提升首屏加载速度、优化SEO表现，并提供更为流畅的用户体验。Vike 作为轻量级且灵活的SSR框架，能有效解决传统客户端渲染模式下的性能瓶颈^{[1][3]}。
- 状态管理：Pinia。作为 Vue 生态中新一代的状态管理库，Pinia 以其直观的API、类型安全支持以及模块化设计，确保了前端数据流的清晰与可维护性，简化了组件间的数据共享与管理^[2]。
- Web 应用框架：Hono。后端采用极速、轻量级的 Web 应用框架 Hono，其针对边缘环境的优化设计和卓越的性能表现，为构建高性能、可扩展的API服务提供了坚实基础，尤其适用于 Serverless 和边缘计算场景^[5]。
- RPC 框架：tRPC。为实现前后端之间的高度类型安全通信，我们引入了tRPC。tRPC 允许前端直接调用后端函数，并在编译时进行端到端类型检查，极大减少了API集成过程中的错误，显著提升了开发效率与代码质量^[4]。
- ORM：Prisma。作为现代化的下一代 ORM，Prisma 提供了直观的数据模型定义、自动生成的类型安全查询以及强大的迁移工具，简化了数

数据库操作，提高了数据访问层的开发效率和健壮性^[6]。

➤ **内存数据库：Redis。**引入 Redis 作为内存数据库，主要用于会话管理和数据缓存，以提升系统的读写速度和响应性能，减轻主数据库的压力，从而优化整体用户体验^[7]。

➤ **打包器：Vite。**前端开发工作流采用 Vite 作为打包工具，其基于 ESM 的开发服务器提供了极速的冷启动和热模块更新，显著加快了开发迭代速度，提升了开发效率^[12]。

➤ **验证库：Zod。**针对数据验证，我们选择 Zod，这是一个 TypeScript 优先的模式声明和验证库。它确保了数据的完整性和类型安全，无论是前端表单输入还是后端API请求，都能得到严格校验，增强了应用的健壮性和安全性^[8]。

➤ **原子化 CSS 库：UnoCSS。**样式解决方案选用 UnoCSS，它是一种高性能、按需生成原子化CSS的引擎。通过最小化CSS文件体积，避免了样式膨胀问题，同时提供了高度灵活的样式和主题定制能力，优化了前端性能^[11]。

本项目的技术栈选择体现了当前 Web 开发领域的前沿趋势，旨在通过前后端分离、服务端渲染、类型安全、高性能以及优化的开发体验，构建一个功能完善、运行流畅且易于维护的 To-Do List 应用。同时，在开发过程中将充分考虑 Web 安全的重要性，采取相应的防护措施，以确保应用的数据安全和用户隐私。

1.2 项目目标

本项目旨在达成以下具体目标：

- **搭建一个完整的 Web 应用：**成功构建一个具备基本任务增删改查功能、用户认证以及数据持久化的 To-Do List 全栈 SSR Web 应用，并确保其不同设备上的响应式表现^[10]。
- **熟悉并掌握SSR的前后端统一思维模型的开发流程：**深入理解基于Vue + Vike（前端 SSR）和 Hono + tRPC（后端 API）的现代化前后端统一思维模型开发模式，掌握从需求分析、系统设计、编码实现到测试部署的完整流程。
- **强化数据安全意识 and 实践能力：**在应用开发中，实践数据输入验证（利

用 Zod)、用户身份认证与权限管理(结合 Hono 和 tRPC),并考虑常见的Web安全漏洞(如 SQL 注入、XSS、CSRF)的防范措施与思路,提升实际安全编码能力。

- 理解 Vue 组件化开发思想与 SSR 机制:深入学习 Vue 的组件化开发范式,并通过 Vike 框架实践 Vue 应用的服务器端渲染,理解其工作原理及对性能等积极影响。
- 实践后端服务接口开发与数据库交互:掌握使用 Hono 框架构建高效、类型安全的 RESTful API 或 tRPC 服务,并通过 Prisma ORM 与 PostgreSQL 数据库(或兼容数据库)进行高效、可靠的数据交互,实现数据的增删改查操作。
- 运用先进的开发工具与库提升效率:熟练运用 Vite 进行项目打包与开发服务器管理,利用 UnoCSS 实现按需原子化 CSS,并通过 Pinia 进行高效的前端状态管理,提升整体开发效率和项目维护性。

二、功能需求分析

2.1功能需求分析

2.1.1 用户认证与授权

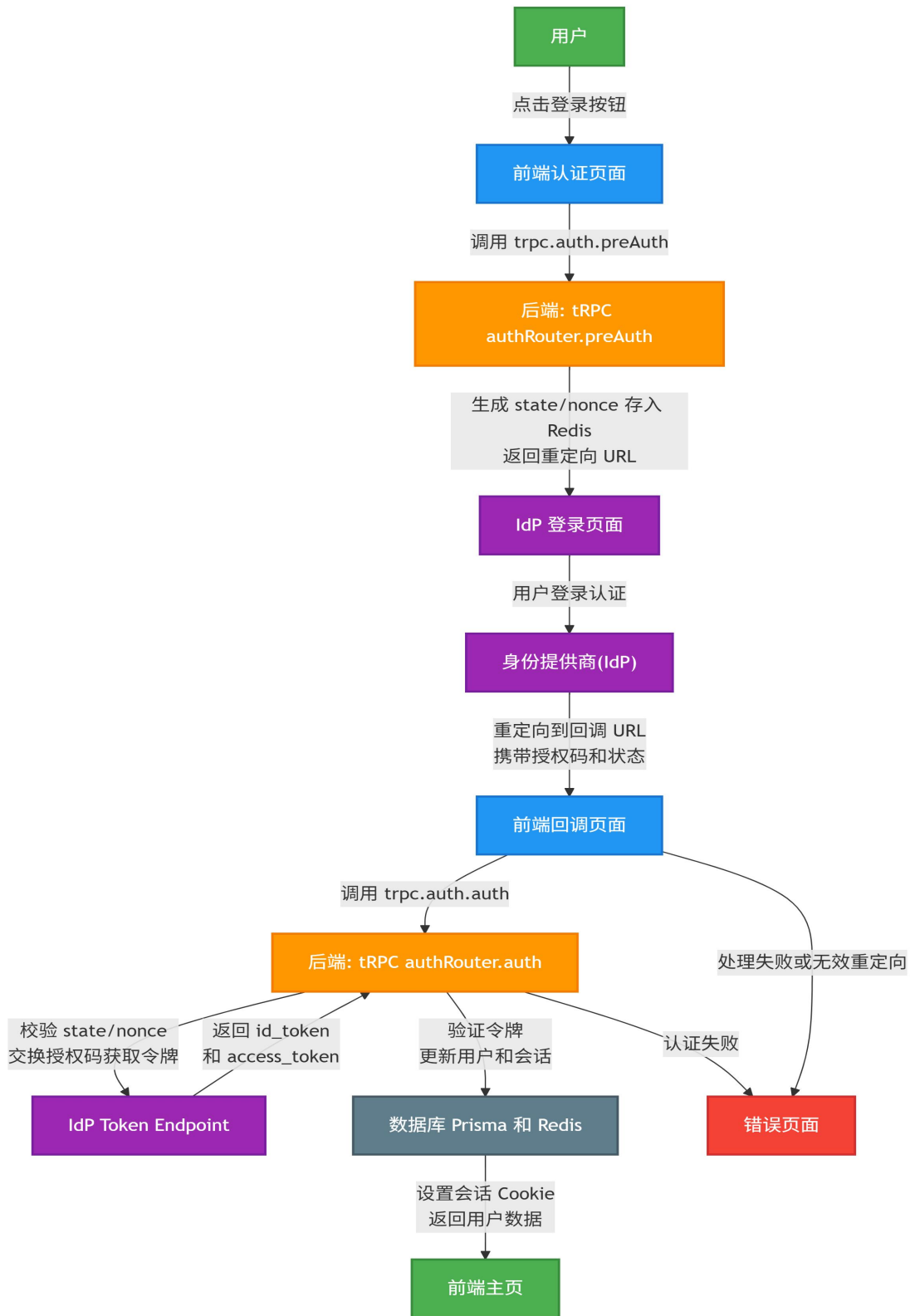


图2- 1用户认证流程图

2.1.1.1 多因子认证支持 (OIDC 集成)

➤ 经 `src/app/components/AuthHandler.vue` 和 `src/app/components/AuthPreHandl`

er.vue, 应用提供了灵活的登录入口, 支持通过环境变量配置与 OpenID Connect (OIDC) 兼容的身份提供商进行集成认证^[9]。

- `src/app/server/plugin/oidc-auth-provider/provider.ts` 实现了 `AuthProvider` 接口, 负责处理 OIDC 认证流程:
 - ✓ 预认证 (`handlePreAuth`): 在用户点击登录按钮后, 根据 OIDC 提供商的配置, 生成 `state` 和 `nonce`, 并在 `AuthProvider` 的实现中构造认证跳转 URL, 引导用户至身份提供商进行认证。相关会话信息 (`state`, `nonce`) 会被存储在 `Redis` 中, 确保认证请求的有效性和安全性。
 - ✓ 认证回调 (`handleAuth`): 身份提供商认证成功后, 会将用户重定向回应用的 `/auth/callback` 页面。`src/app/components/AuthHandler.vue` 会捕获 URL 中的 `code` 和 `state` 参数, 并将其传递给后端 `tRPC` 接口。
 - ✓ 后端 `tRPC` `authRouter.auth` (`src/app/server/trpc/routers/auth.ts`) 接收到回调后, 会从 `Redis` 中校验 `state` 和 `nonce` 的匹配性, 并使用 `code` 向身份提供商交换 `id_token` 和 `access_token`。
 - ✓ ID Token 验证与用户会话管理: 后端会利用 `jose` 库对 `id_token` 进行严格验证 (包括签名、有效期、`Issuer`、`Audience` 以及 `Nonce`), 确保其真实性和完整性。验证成功后, 从 `id_token` 中提取用户信息 (如 `sub`, `name`, `email` 等), 用于在 `Prisma` (`src/app/server/db/prisma.ts`) 中创建或更新本地用户账户。
 - ✓ 用户会话信息 (如 `userId`, `idToken`) 将安全地存储于 `Redis` 中, 并关联一个 `sessionId`。此 `sessionId` 会通过 HTTP Only 的 Cookie (`src/app/server/utils/http-helpers.ts`) 返回给客户端, 用于后续的身份验证。
- 登出功能 (`handleLogout`): 用户登出时, 后端会根据 `sessionId` 从 `Redis` 中获取 `idToken`, 并向 OIDC 提供商发起登出请求, 同时清除服务器端和客户端的会话信息, 确保安全退出。

2.1.1.2 认证状态管理

- `src/app/stores/auth.ts` `Pinia Store` 用于管理认证相关的错误信息 (`isError`, `authMethod`), 并在客户端进行持久化存储 (通过 `sessionStorage`)。
- `src/app/pages/auth/callback/+guard.ts` 确保已登录用户无法再次访问认证页

面，直接重定向到主页。

2.1.2 任务管理

2.1.2.1 任务创建

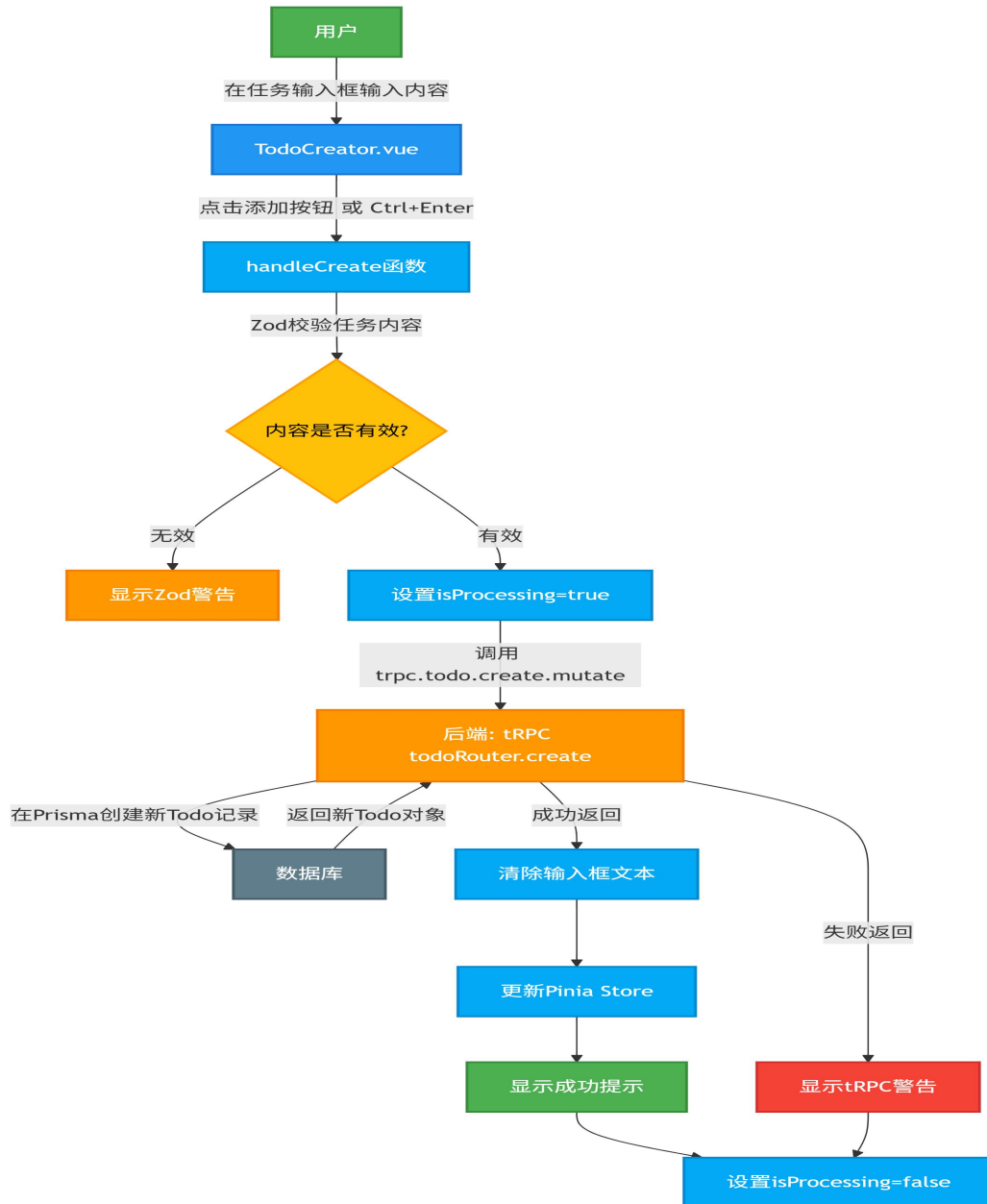


图2- 2任务创建流程图

- 用户可以在 `TodoCreator.vue` 组件中输入任务内容，并通过点击“添加”按钮或使用 `Ctrl+Enter` 快捷键快速创建新任务
- 任务内容在提交前会通过 `zod` 的 `TodoDataSchema (src/shared/schema/misc.ts)` 进行客户端及服务端的严格校验，确保任务内容不为空且长度符合规范。

- 成功创建后，前端会清空输入框，并通过 `upsertTodos` 方法更新 Pinia 中的任务列表，同时显示成功提示。

2.1.2.2 任务列表展示

- `TodoList.vue` 负责渲染所有任务项，每个任务由 `TodoListItem.vue` 组件展示。
- 任务列表支持按照是否星标 (`isImportant`) 和更新时间 (`updatedAt`) 进行排序，确保用户能优先看到重要且最新的任务。
- 任务列表能够根据用户设置 (`isShowCompleted useProfileStore`) 动态显示或隐藏已完成任务，通过 `CompletedTodoToggler.vue` 进行控制。

2.1.2.3 任务详情与编辑



图2- 3任务更新流程图

- 快速编辑: 在 `TodoListItem.vue` 中, 用户可以直接点击任务文本进行内联编辑, 修改任务内容。

- 详细编辑：点击任务项会触发 `TodoSidebar.vue` 的显示，侧边栏会加载并展示选中任务的详细信息，包括：
 - ✓ 任务内容 (`text`)：可编辑。
 - ✓ 任务描述 (`description`)：可编辑，支持多行文本。
 - ✓ 完成状态 (`isCompleted`)：通过复选框或侧边栏按钮切换。
 - ✓ 重要性 (`isImportant`)：通过星标图标或侧边栏按钮切换。
 - ✓ 截止日期 (`deadline`)：可添加、修改或清除截止日期，支持快捷设置“今天”、“三天内”、“一周内”等预设日期，并使用 `VueDatePicker` 组件提供日期选择器。
- 所有修改都将通过 `todo.update` 接口进行类型安全的更新操作，并实时反映在UI上。
- 截止日期提醒：`TodoListItem.vue` 根据任务的截止日期和当前时间，动态显示时钟图标并改变颜色和动画，以提示任务的紧急程度（如过期、临近）。

2.1.2.4 任务状态操作

- 完成/未完成：通过任务列表项中的复选框或侧边栏按钮快速切换任务的完成状态。
- 标记重要/不重要：通过任务列表项中的星标图标或侧边栏按钮快速切换任务的重要性。
- 删除任务：任务列表项和侧边栏均提供删除按钮，通过 `todo.delete` 接口从数据库中删除任务，并实时更新前端列表。

2.1.2.5 批量操作

- 应用支持进入多选模式 (`isInMultiSelectMode`)，用户可以批量选择任务。
- `TodoBatchTools.vue` 提供了一系列批量操作工具，包括：
 - ✓ 全选/取消全选。
 - ✓ 批量标记为完成/未完成/反转完成状态。
 - ✓ 批量标记为重要/不重要/反转重要状态。
 - ✓ 批量删除选中的任务。
- 这些批量操作通过 `todo.update` 或 `todo.delete` 接口高效地更新或删除多个

任务。

2.1.3 用户界面与体验

2.1.3.1 响应式布局 (Sidebar.vue)

- 应用界面采用响应式设计，适应不同屏幕尺寸（特别是桌面端和移动端）。
- Sidebar.vue 实现了可拖拽调整宽度的侧边栏，支持自由模式（浮动）和相对模式（固定），在移动设备上默认折叠并提供全屏覆盖，优化了小屏幕设备的交互体验。

2.1.3.2 主题切换 (ThemeSwitcher.vue)

- 用户可以方便地切换应用的主题（亮色/暗色模式），主题选择会被持久化到Cookie中，确保下次访问时保持一致。

2.1.3.3 全局提示 (Toast.vue, useToastStore)

- 通过 Toast.vue 组件和 useToastStore，应用提供了统一的、非侵入性的全局通知机制，用于显示操作成功、警告和错误信息，提升用户反馈的及时性。

2.1.3.4 键盘快捷键 (MagicKeys.vue, Key.vue)

- MagicKeys.vue 组件利用 @vueuse/core 库，为关键操作（如创建任务的 Ctrl+Enter）提供了键盘快捷键支持，提高了高级用户的操作效率。

2.1.3.5 加载指示器 (Loading.vue)

- 在数据加载或处理过程中，显示加载动画 (Loading.vue)，向用户提供视觉反馈，避免不必要的等待焦虑。

2.1.3.6 通用UI组件

- 包括自定义的 Button、Input、Textarea、Checkbox、Toggler、Icon、DropdownMenu、Picker 等组件，保证了应用界面的风格统一性和可复用性。

2.1.4 数据管理与持久化

2.1.4.1 Pinia 状态管理

- `src/app/stores` 目录下定义了 `auth`, `profile`, `sidebar`, `toast`, `todo` 等多个 Pinia Store，实现了应用数据的集中管理和响应式更新。

2.1.4.2 Prisma ORM (`src/app/server/db/prisma.ts`)

- 后端使用 Prisma 作为 ORM，提供了类型安全的数据库操作接口，简化了与 PostgreSQL 数据库的交互。
- `src/shared/schema/prisma.ts` 定义了与数据库模型对应的 Zod Schema (`UserSchema`, `AccountSchema`, `TodoSchema`)，确保数据在应用各层之间的类型一致性。

2.1.4.3 Redis 缓存与会话管理 (`src/app/server/db/redis.ts`)

- Redis 被用于存储用户会话信息（例如 OIDC 认证流程中的 `state` 和 `nonce`）以及用户登录会话，提高了认证和数据访问的速度。

2.1.5 前后端通信与验证

2.1.5.1 tRPC 远程过程调用

- 前端通过 `@trpc/client` (`src/app/server/trpc/client.ts`) 调用后端 tRPC 服务，实现了端到端类型安全的 API 请求。
- 后端 tRPC 路由 (`src/app/server/trpc/routers/_app.ts`) 定义了 `authRouter`, `todoRouter`, `userRouter`，提供了明确的 API 接口和输入/输出类型。
- `src/app/server/trpc/server.ts` 定义了 `publicProcedure` 和 `authedProcedure`，其中 `authedProcedure` 强制要求用户登录才能访问，实现了基础的权限控制。

2.1.5.2 Zod 数据验证

- `zod` (`src/shared/schema`) 在前端和后端都用于数据结构定义和运行时验证，确保了数据传输和处理的健壮性。例如，任务创建和更新时的数据都必须符合 `TodoDataSchema` 的定义。

2.1.5.3 Hono Web 应用框架 (`src/app/server/app.ts`)

- Hono 作为轻量级 Web 框架，负责处理 HTTP 请求，并将 `/api/trpc` 路由的请求转发给 tRPC 处理器。

- 内置了CORS (hono/cors) 和CSRF (hono/csrf) 中间件，增强了API的安全性，防止跨域请求和跨站请求伪造攻击。

2.2 非功能需求

非功能需求 (Non-Functional Requirements, NFRs) 描述了系统如何执行其功能，包括性能、安全性、可维护性等方面的技术要求。这些需求对于确保系统质量、用户满意度和长期可持续性至关重要。本项目在技术选型和架构设计阶段充分考虑了以下非功能需求：

2.2.1 性能要求

本项目旨在提供流畅、响应迅速的用户体验，对系统性能提出了以下要求：

- 响应时间：核心业务操作（如用户登录、任务创建/更新/删除、任务列表加载）的平均响应时间应在 500 毫秒以内，高并发场景下响应时间不应超过 2 秒。
- 并发处理能力：系统应能支持至少 100 个并发用户，并保持稳定的性能表现。
- 加载速度：
 - ✓ 首屏加载：借助 Vue + Vike (SSR) 技术，应用的首屏加载时间应控制在2秒以内，有效改善用户初次访问体验和搜索引擎抓取效率。
 - ✓ 数据加载：通过 Redis 进行会话管理和热点数据缓存，减少数据库I/O，确保任务列表及详情页数据加载迅速。
- 资源利用率：服务器CPU和内存利用率在正常负载下应保持在合理范围（如CPU低于 70%，内存低于 80%），避免资源瓶颈。
- 打包优化：采用 Vite 作为打包器，实现按需编译和代码分割，优化前端资源加载，并通过 UnoCSS 自动生成原子化 CSS，最大限度减少 CSS 体积，提升页面渲染速度。

2.2.2 安全要求

数据安全和用户隐私是本项目关注的重点，系统应满足以下安全要求：

- 用户身份认证：
 - ✓ 采用 OIDC (OpenID Connect) 协议进行身份认证，利用成熟的第三方

身份提供商确保认证流程的安全性、可靠性和可扩展性。

- ✓ 认证过程中严格遵循OIDC规范，包括 `state`, `nonce` 参数校验、`id_token` 签名验证和有效期检查，防止认证重放攻击和中间人攻击。
- ✓ 用户会话信息通过 `Redis` 进行管理，并使用加密的 `sessionId` 通过 `HTTP Only Cookie` 进行传输，防止客户端脚本窃取。
- 数据传输安全：
 - ✓ 前后端所有通信（包括 `tRPC` 请求）应通过 `HTTPS` 协议进行加密传输，防止数据窃听和篡改。
 - ✓ `tRPC` 提供了端到端的类型安全保障，减少了因类型不匹配导致的安全漏洞风险。
- 数据存储安全：
 - ✓ 确保数据库访问权限最小化，仅允许必要的后端服务进行操作。
- 输入验证与防范：
 - ✓ 利用 `Zod` 在客户端和服务端对所有用户输入进行严格的数据格式和业务逻辑验证与 `Prisma` 处理层提供的自动 `SQL` 验证，防止 `SQL` 注入、`XSS (Cross-Site Scripting)` 等常见 `Web` 攻击。
 - ✓ 后端 `Hono` 框架内置 `CORS (Cross-Origin Resource Sharing)` 和 `CSRF (Cross-Site Request Forgery)` 中间件，有效防止跨域资源访问滥用和伪造请求。
- 权限控制： 后端 `tRPC` 通过 `authedProcedure` 过程确保只有经过认证的用户才能访问受保护的API接口，实现基础的权限控制。

2.2.3 可维护性要求

本项目注重代码质量和可维护性，以便于团队协作、功能扩展和长期迭代：

- 代码规范： 遵循统一的编码规范和风格指南，确保代码的可读性和一致性。
例如，`Vue` 组件的单文件组件结构、`TypeScript` 的严格类型定义。
- 模块化设计：
 - ✓ 前端采用 `Vue` 组件化 开发，将UI划分为独立的、可复用的组件，降低耦合度。

- ✓ 前端状态管理使用 **Pinia**，将不同模块的状态分离，并通过明确的 **actions** 和 **getters** 管理数据流，提高状态管理的清晰度。
- ✓ 后端采用 **tRPC** 定义清晰的模块化API路由 (**authRouter, todoRouter, userRouter**)，使后端业务逻辑分离且易于理解和测试。
- 类型安全：贯穿整个项目使用 **TypeScript**，并通过 **tRPC** 和 **Zod** 实现了端到端（从数据库**Schema**到前端组件）的类型安全，减少了运行时错误，提高了开发效率和代码质量。
- 文档与注释：关键代码逻辑、复杂算法、API 接口等应有详细的注释，并提供必要的文档说明，方便新成员快速上手和问题排查。
- 测试：编写单元测试、集成测试和端到端测试，确保功能正确性和代码变更的稳定性，降低回归风险。
- 依赖管理：合理管理项目依赖，定期更新依赖库，及时修复安全漏洞和利用新特性。

2.2.4 可扩展性要求

系统架构应具备良好的可扩展性，以应对未来业务增长和功能迭代的需求：

- 水平扩展：
 - ✓ 前端 **Vue + Vike (SSR)** 应用可以轻松进行多实例部署，通过负载均衡器实现水平扩展。
 - ✓ 后端 **Hono** 框架轻量且高效，易于在**Serverless**环境或容器化部署中进行横向扩展，满足高并发场景需求。
 - ✓ **Prisma ORM**和 **Redis** 数据库本身支持集群部署和读写分离，为数据层的扩展提供了可能。
- 功能扩展：
 - ✓ 清晰的 **tRPC API** 设计和模块化路由结构，使得添加新功能或修改现有功能时，对现有代码的影响最小。
 - ✓ 前端 **Vue** 组件化 和 **Pinia** 模块化状态管理 使得新增**UI**组件或业务逻辑时，可以独立开发和维护，降低了复杂性。
- 技术栈兼容：所选技术栈均为当前主流且活跃的开源项目，拥有丰富的社区支持和生态系统，便于未来引入新的工具或集成其他服务。

2.2.5 可用性要求

系统应具备良好的用户体验和高可用性：

- 用户界面友好：界面设计直观、操作简单，遵循用户习惯，提供良好的交互反馈。
- 响应式设计：通过 **UnoCSS** 和灵活的布局策略，确保应用在不同尺寸设备（桌面、平板、手机）上均能提供一致且优化的用户体验。
- 错误提示与反馈：通过 **Toast** 组件提供清晰、及时的操作结果反馈和错误提示，引导用户解决问题。
- 键盘可操作性：支持通过键盘快捷键进行常用操作（如任务创建），提高高级用户的操作效率。
- 无障碍性：尽可能考虑无障碍设计原则，确保残障用户也能顺利使用。
- 系统稳定运行时间：核心服务应保证 **99.9%** 的年度运行时间，最大程度减少非计划停机时间。

三、系统架构设计

3.1 架构图设计

本项目的系统总体架构采用分层设计理念，将前端、后端以及数据存储进行明确分离，并通过标准化接口进行通信。这种设计旨在提升系统的可维护性、可扩展性和安全性，同时确保高效的用户体验。

该架构图展示了用户如何通过互联网访问前端 **Web** 应用，前端应用（**Vue.js + Vike**）负责用户界面渲染和与后端服务的交互。后端服务（**Hono**）处理业务逻辑，并通过 **tRPC** 与前端进行类型安全的通信，同时利用 **Prisma ORM** 与 **PostgreSQL** 数据库进行数据持久化，并借助 **Redis** 进行缓存和会话管理。**OIDC** 身份提供商负责用户认证，确保了系统的安全性。

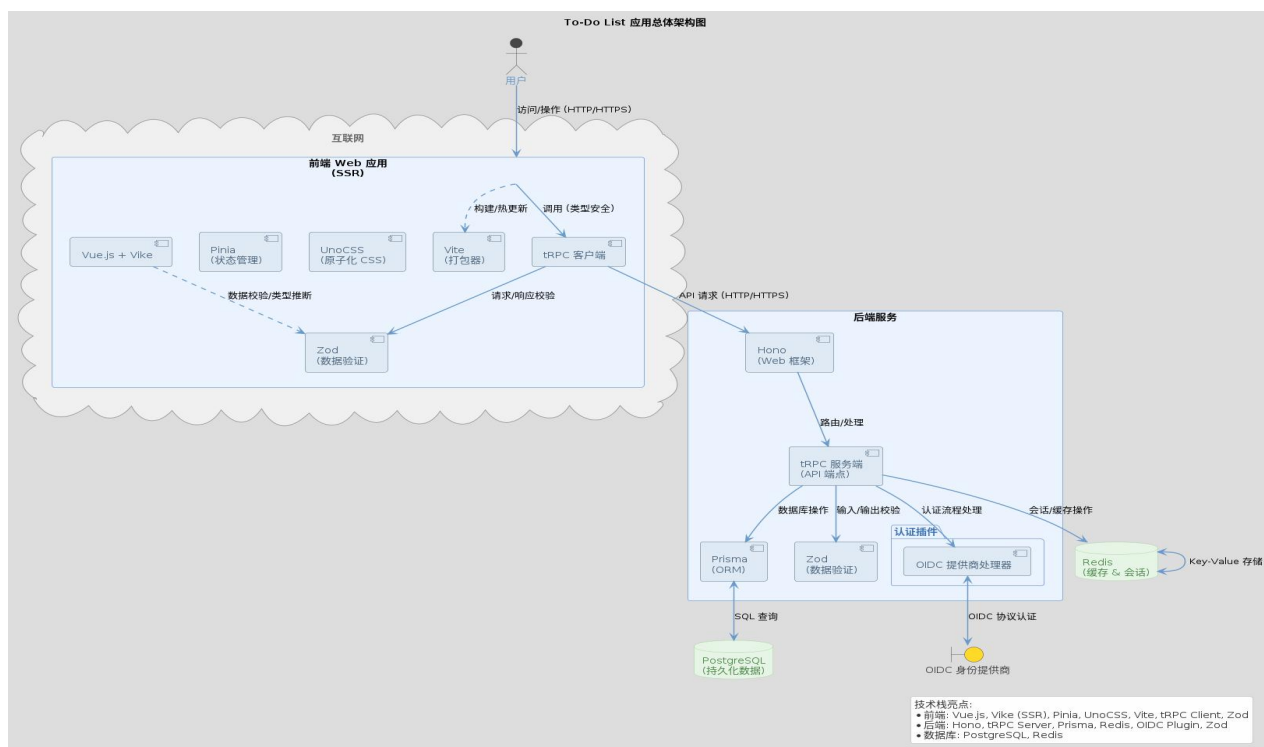


图 3- 1 系统总体架构图

3.2 数据库设计

本项目采用关系型数据库进行数据持久化，并通过 **Prisma ORM** 进行数据操作。数据库设计遵循范式化原则，以确保数据完整性和减少冗余。

3.2.1 数据库 E-R 图

以下是本系统数据库的实体关系 (E-R) 图，它直观地展示了各个实体（如用户、账户、任务）及其之间的关系和属性。此图有助于理解数据库的逻辑结构和数据流。

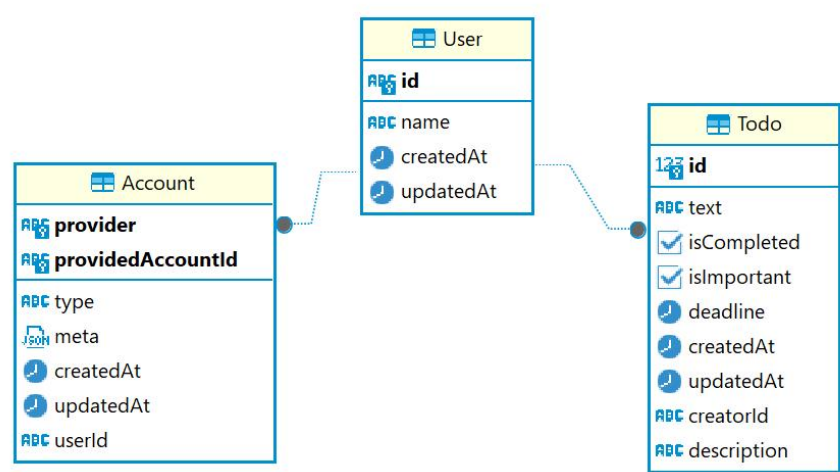


图 3- 2 数据库实体关系图

3.2.2 User 表

User 表用于存储系统中的用户信息。

表 3- 1 User 表

字段	数据类型	是否为主键/外键	备注
id	CUID2 (字 符 串)	是，主键	用户的唯一标识符，Prisma 自动生成
createdAt	DateTime	否	用户创建时间
updatedAt	DateTime	否	用户信息最后更新时间

3.2.2 Account 表

Account 表用于存储用户通过外部身份提供商（如 OIDC）登录的账户信息，实现多账户关联。

表 3- 2 Account 表

字段	数据类型	是否为主键/外键	备注
----	------	----------	----

字段	数据类型	是否为主键/外键	备注
type	String	否	账户类型（例如：OAuth, OIDC）
provider	String	是，复合主键一部分	身份提供商名称（例如：Google, GitHub, My SSO Issuer URL）
providerAccountId	String	是，复合主键一部分	身份提供商侧的用户唯一 ID
meta	Json (可空)	否	存储来自身份提供商的额外元数据（例如：id_token, access_token）
createdAt	DateTime	否	账户创建时间
updatedAt	DateTime	否	账户信息最后更新时间
userId	CUID2（字符串）	是，外键	关联到 User 表的 id 字段

3.2.3 Todo 表

Todo 表用于存储用户的待办事项任务。

表 3-3 Todo 表

字段	数据类型	是否为主键/ 外键	备注
id	Int	是，主键， 自增	任务的唯一标识符
text	String	否	任务的主要内容，非空
description	String (可空)	否	任务的详细描述
isCompleted	Boolean	否	任务是否已完成
isImportant	Boolean	否	任务是否重要
deadline	DateTime (可 空)	否	任务的截止日期和时间
createdAt	DateTime	否	任务创建时间
updatedAt	DateTime	否	任务最后更新时间
creatorId	CUID2 (字 符 串)	是，外键	创建该任务的用户 ID，关联 到 User 表的 id 字段

四、系统实现

本章节将详细阐述 To-Do List 应用在前端、后端和安全层面的具体实现细节。我们将结合实际代码文件，说明关键模块的逻辑流程、技术选型如何落地以及如何保障系统的安全性。为直观展示实现过程，将在各模块前提供相应的 PlantUML 流程图。

4.1 前端实现

前端部分主要负责用户界面的呈现、用户交互逻辑以及与后端服务的通信。

我们基于 Vue 和 Vike 构建了一个高性能的服务器端渲染（SSR）应用。

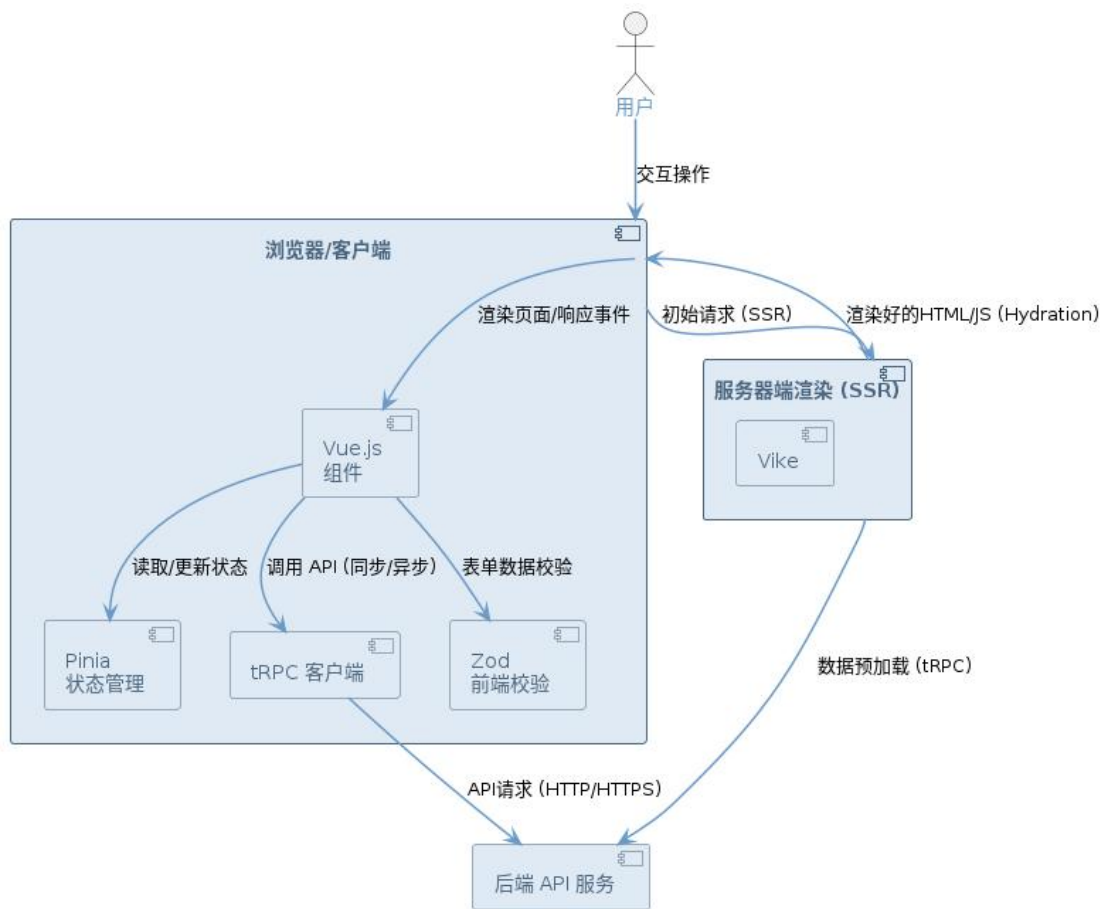


图 4-1 前端整体实现流程图

4.1.1 页面结构与布局设计

应用整体采用响应式布局，以适应不同设备。核心布局由 `src/app/layouts/Root.vue` 定义，它提供了页面的基础结构和全局组件（如 Toast）。认证相关的页面使用 `src/app/layouts/Auth.vue` 布局，而任务管理页面则使用 `src/app/layouts/ToDo.vue`，该布局包含了可折叠和拖拽的侧边栏。

页面结构与布局设计通过组件的组合和嵌套，实现了清晰的职责划分和高度的复用性，例如：

- 根布局 (Root.vue)：作为应用的入口布局，它引入了 UnoCSS (`virtual:uno.css`) 和全局主题样式 (`../assets/theme.css`)，并负责渲染全局的 Toast 提示组件 (`../components/Toast.vue`)，为所有页面提供一致的视觉和通知体验。
- 认证布局 (Auth.vue)：专为登录、注册等认证流程设计，通过简单的 div 居中内容，提供简洁的用户认证界面。

- 任务布局 (Todo.vue): 此布局是任务管理界面的核心，它包含一个可响应式调整的侧边栏 (../components/ToDoSidebar.vue) 和主要内容区域。在移动端，侧边栏可被隐藏，并通过菜单按钮 (i-mdi:menu) 切换显示状态。

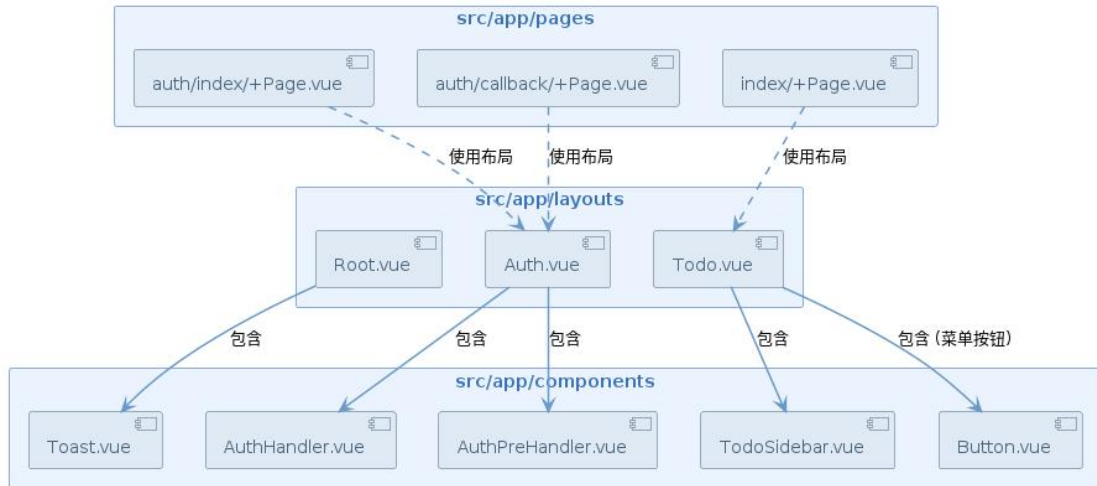


图 4-2 前端布局与页面组件关系图

4.1.2 路由与组件划分

项目路由通过 Vike 框架的文件系统路由自动生成。每个页面都由一个 +Page.vue 文件定义，并可以配置独立的布局 (+config.ts)、数据获取 (+data.ts) 和访问守卫 (+guard.ts)。这种约定优于配置的方式极大地简化了路由管理。

- 认证路由 (src/app/pages/auth):
 - ✓ src/app/pages/auth/index/+Page.vue：负责展示登录选项，并集成 AuthPreHandler.vue 用于发起 OIDC 认证前的预处理。
 - ✓ src/app/pages/auth/callback/+Page.vue：作为 OIDC 认证回调页面，集成 AuthHandler.vue 来处理认证后的逻辑，包括令牌交换和用户会话建立。
 - ✓ 认证页面的访问守卫 (src/app/pages/auth/callback/+guard.ts) 确保已登录用户不会重复进入认证流程。
- 主页路由 (src/app/pages/index):
 - ✓ src/app/pages/index/+Page.vue：应用的主任务列表页面，此处会包含任务的展示和操作组件。
 - ✓ src/app/pages/index/+data.ts：用于在服务器端渲染时预加载任务数据并通过 store 传递给任务列表组件进行渲染，提升首屏加载速度并避免闪烁。

组件划分严格遵循 Vue 的最佳实践，将功能内聚的模块封装成独立的组件，例如：

- 任务相关组件：TodoCreator.vue (创建任务), TodoList.vue (任务列表), TodoListItem.vue (单个任务项), TodoSidebar.vue (任务详情/编辑侧边栏), TodoBatchTools.vue (批量操作工具)。
- 通用UI组件：Button.vue, Input.vue, Checkbox.vue, Icon.vue, DropdownMenu.vue, Picker.vue, Loading.vue, ThemeSwitcher.vue。
- 表单渲染组件：json-form 目录下的 JSONForm.vue 及其 renderers，实现了基于 JSON Schema 动态渲染表单字段的能力，提高了表单的通用性和可配置性，特别用于认证流程中的动态表单需求。

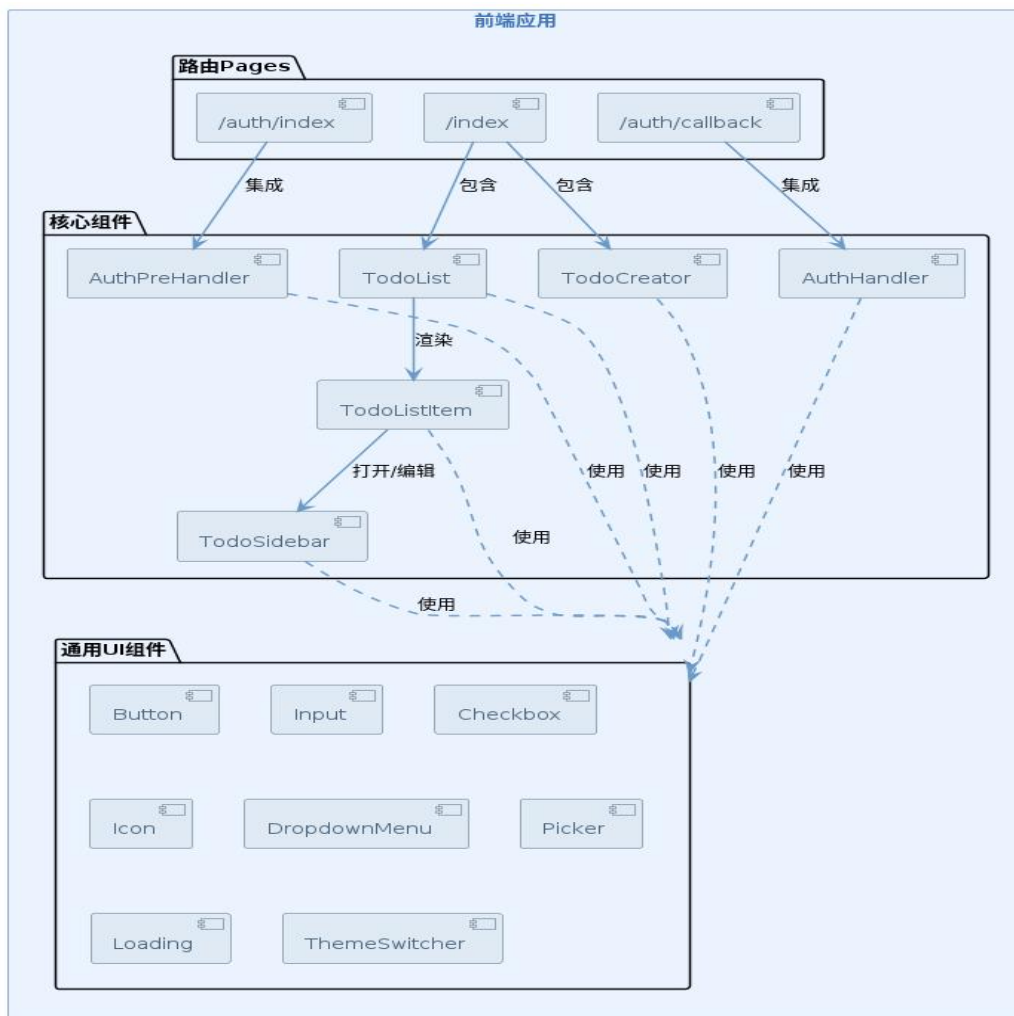


图 4-3 前端路由与核心组件交互流程图

4.1.3 状态管理 (Pinia)

前端状态管理采用 Pinia，它以其简洁的 API、强大的类型推断能力和模块化设计，完美契合了 Vue 3 Composition API。项目中定义了多个 Pinia Store 来管理不同模块的状态：

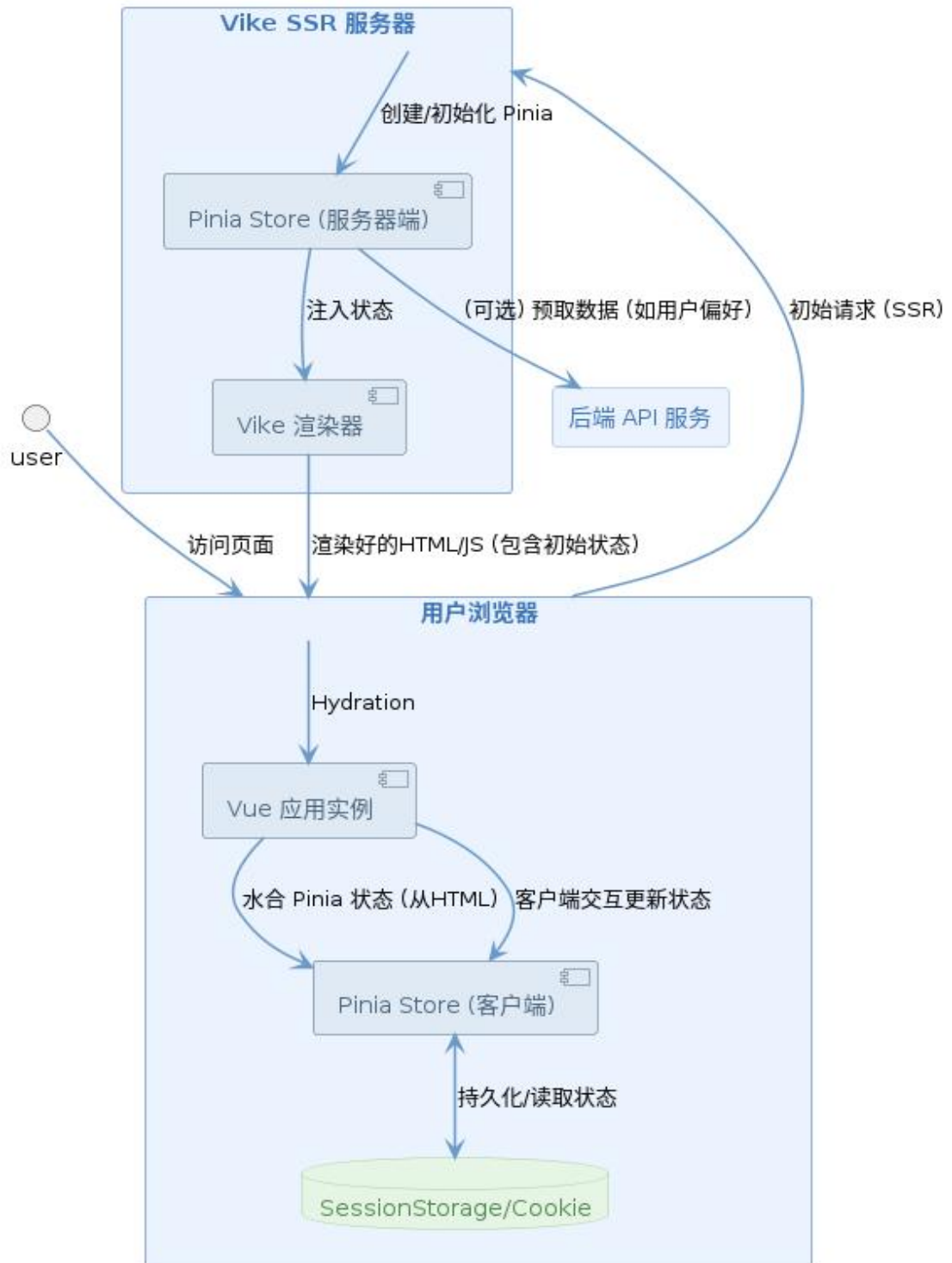


图 4-4 Pinia 状态管理与 SSR 流程图

- `useAuthStore (src/app/stores/auth.ts)`: 管理用户认证状态, 包括登录错误信息 (`isError`) 和当前认证方法 (`authMethod`)。 `authMethod` 通过 `sessionStorage` 进行持久化, 以在认证流程中保持状态。
- `useProfileStore (src/app/stores/profile.ts)`: 管理用户偏好设置, 如主题 (`theme`) 和是否显示已完成任务 (`isShowCompleted`)。这些设置通过 `Cookie` 进行持久化 (`src/app/renderer/+htmlAttributes.ts, src/app/stores/profile.ts`)。
- `useSidebarStore (src/app/stores/sidebar.ts)`: 控制侧边栏的显示/隐藏状态 (`isFolding`) 和宽度 (`width`), 并处理响应式布局下的侧边栏行为。
- `useToastStore (src/app/stores/toast.ts)`: 集中管理全局的 `Toast` 通知, 提供 `info, warn, error` 方法用于显示不同类型的消息, 并处理消息的自动消失逻辑。
- `useTodoStore (src/app/stores/todo.ts)`: 管理所有待办事项数据, 包括任务列表的增删改查 (`upsertTodos, deleteTodos`)、任务的选择状态 (`selectedTodos, pickedTodoId`) 和多选模式 (`isInMultiSelectMode`)。任务列表在这里也实现了基于重要性和更新时间的排序逻辑。

Pinia 的集成通过 Vike 的生命周期钩子实现:

- 服务器端 (`src/app/pages/auth/index/+onCreatePageContext.server.ts, src/app/pages/auth/index/+onCreateApp.server.ts, src/app/pages/auth/index/+onAfterRenderHtml.ts`): 在服务器端创建 `Pinia` 实例并进行状态初始化 (特别是从 `Cookie` 读取用户偏好), 然后将初始化后的状态注入 `HTML`, 以支持 `SSR`, 使主题不会在客户端和服务端间偏移。
- 客户端 (`src/app/pages/auth/index/+onCreateApp.client.ts`): 在客户端 `Hydration` 阶段, 重新创建 `Pinia` 实例并使用服务器端注入的状态进行水合, 确保前后端状态一致。

4.1.4 API 调用与数据渲染

前端通过 `tRPC` 客户端 (`src/app/server/trpc/client.ts`) 与后端 `API` 进行交互。`tRPC` 提供了出色的类型安全特性, 使得前端对后端接口的调用如同调用本地函数般简单且不易出错。数据渲染则由 `Vue` 组件负责, 通过响应式数据绑定实现页面的动态更新。

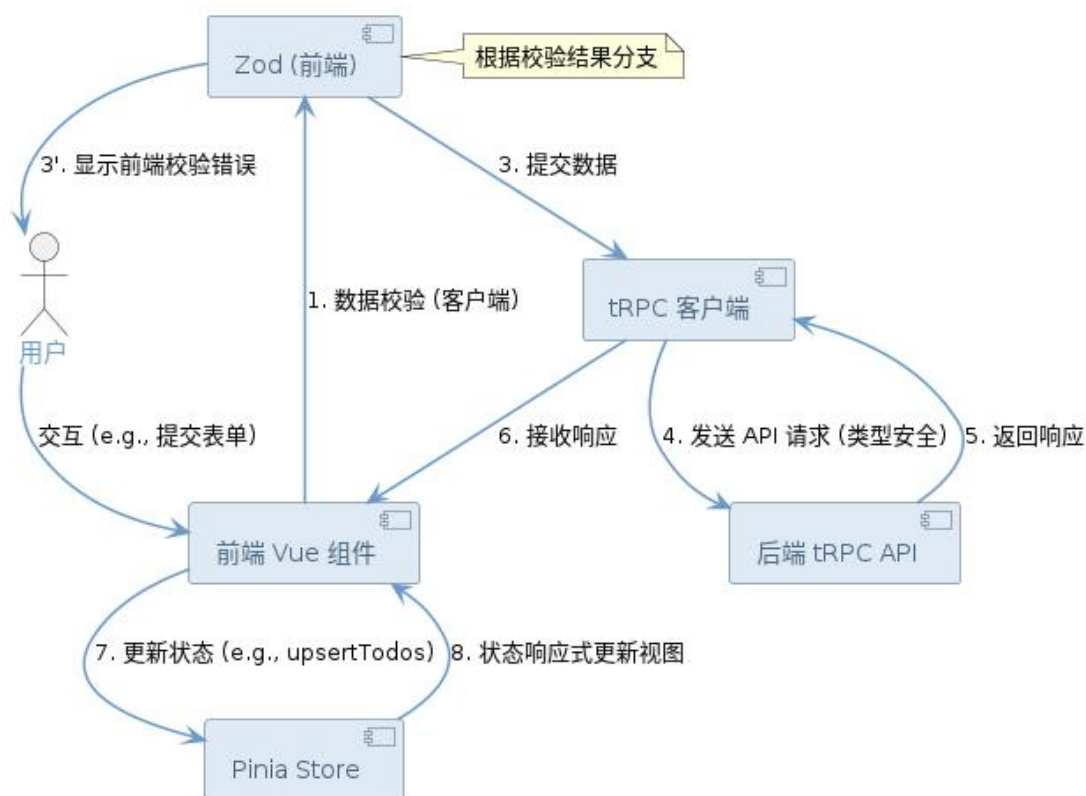


图 4-5 前端 API 调用与数据渲染流程图

- tRPC 客户端集成：前端导入 trpc 实例 (`import { trpc from '@server/trpc/client';`)，可以直接调用后端定义的 API 接口，例如 `todo.create` 用于创建任务，`todo.listOwned` 用于获取用户任务列表。
- 数据获取与更新：
 - ✓ 服务器端数据预加载：利用 Vike 的 `+data.ts` 与 `+onData.ts` 钩子，可以在服务器端预先获取数据（例如 `src/app/pages/index/+data.ts` 中获取用户待办事项），并将数据作为 `props` 传递给 Vue 组件，实现数据的同构渲染，减少客户端首次加载的数据请求。
 - ✓ 客户端实时更新：当用户进行任务创建、更新、删除等操作时，前端组件会调用相应的 tRPC `mutate` 方法，并在成功响应后，通过 Pinia Store 的 `actions` (`upsertTodos`, `deleteTodos`) 更新本地状态，从而实时反映在用户界面上。
- 表单数据绑定与验证：Vue 的 `v-model` 用于双向绑定表单输入 (`Input.vue`, `Textarea.vue`)。在数据提交到后端前，会通过 Zod (`src/shared/schema/misc.ts`) 进行客户端验证，提供即时反馈，例如在 `TodoCreator.vue` 中对任务文本的

长度和非空校验。

4.1.5 任务创建模块实现

任务创建功能主要通过 `TodoCreator.vue` 组件实现，该组件负责任务内容的输入、客户端校验以及调用后端 API 创建新任务。

示例代码： `TodoCreator.vue` (核心逻辑片段)：

```
<script setup lang="ts">
const { info, warn, zWarn } = useToastStore();
const { upsertTodos } = useTodoStore();
const text = ref("");
const isProcessing = ref<boolean>(false);
const handleCreate = async () => {
  if (isProcessing.value) return;
  TodoDataSchema.omit({
    id: true,
  })
  .parseAsync({
    text: text.value,
  })
  .then(async ({ text: pText }) => {
    isProcessing.value = true;
    await trpc.todo.create
      .mutate({
        text: pText!,
      })
      .then((todo) => {
        text.value = "";
        upsertTodos(todo);
        info("成功创建任务");
      })
  })
}
```

```

        .finally(() => (isProcessing.value = false));
    })
    .catch(zWarn);
};
</script>
<template>
  <div class="flex gap-1 w-full items-center">
    <Input v-model="text" full-width icon="i-mdi:book" placeholder="任务内容" />
    <Button
      :is-processing
      icon="i-mdi:plus"
      magic-key="Control+Enter"
      @click="handleCreate"
      @magic-click="handleCreate"
    />
  </div>
</template>

```

示例界面截图：任务创建界面



图 4-6 任务创建界面图

4.2 后端实现

后端服务使用 Hono 框架构建，并集成了 tRPC 提供类型安全的 API 接口。

数据层通过 Prisma ORM 和 Redis 缓存进行高效管理，确保了服务的稳定性、性能和可扩展性。

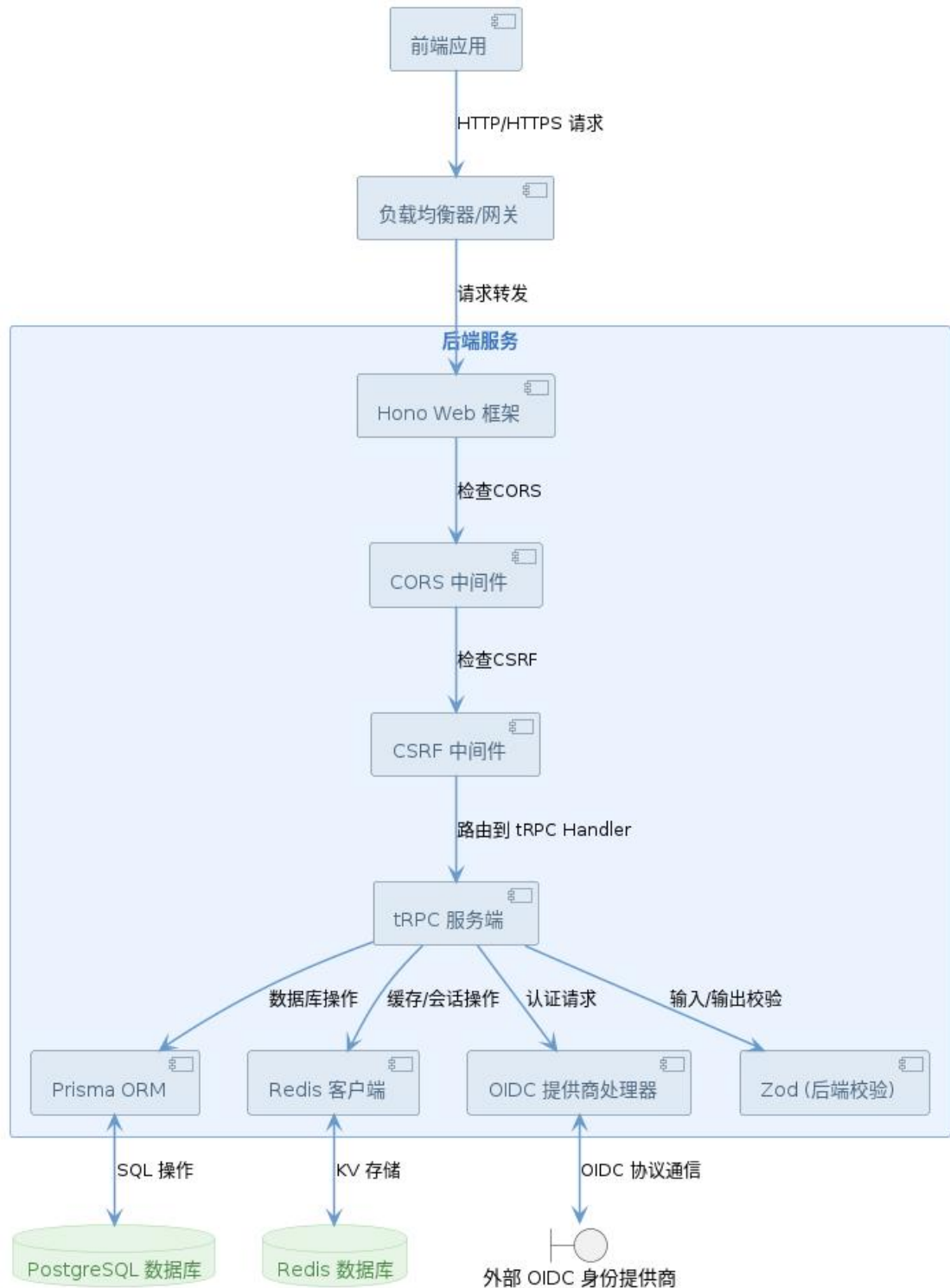


图 4-7 后端整体实现流程图

4.2.1 路由与中间件配置

后端应用的核心入口是 `src/app/server/app.ts`，它基于 Hono 框架进行构建，负责 HTTP 请求的路由和中间件管理。Hono 提供轻量且高性能的 Web 框架特性，能够高效地处理传入的 HTTP 请求。

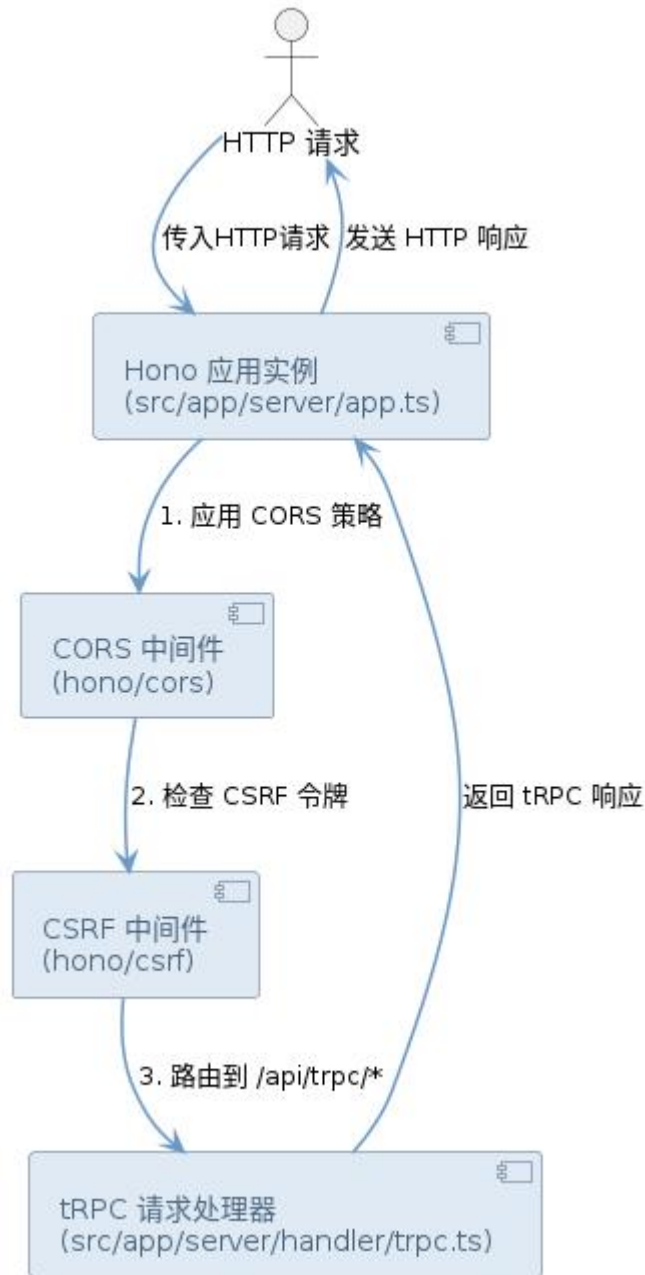


图 4-8 后端路由与中间件处理流程图

- **tRPC 路由挂载：** Hono 应用将所有 `/api/trpc/*` 路径下的请求路由到 `trpcHandler (src/app/server/handler/trpc.ts)`，该处理器由 tRPC 的 `fetchRequestHandler` 创建，负责解析 tRPC 请求、调用相应的后端过程并返回结果。

- **CORS 配置：**通过 `hono/cors` 中间件进行跨域资源共享配置，允许前端应用安全地访问后端 API。配置中明确指定了允许的 `origin` (来自 `PUBLIC_ENV__URL`)、`allowMethods` (GET, POST, OPTIONS) 和 `credentials` (允许发送 Cookie)，确保了安全且灵活的跨域访问。
- **CSRF 防护：**集成 `hono/csrf` 中间件，对请求进行 CSRF 令牌验证，防止跨站请求伪造攻击。配置中列出了合法的 `origin`，增加了系统的安全性。

4.2.2 tRPC 接口定义与认证过程实现

后端 API 接口通过 tRPC 的 `router` 和 `procedure` 概念进行定义。其中，`authedProcedure` 是一个自定义的高阶过程，用于确保只有经过认证的用户才能调用受保护的 API。这是后端访问控制的核心实现。

示例代码：src/app/server/trpc/server.ts 中 `authedProcedure` 定义：

```
export const authedProcedure = t.procedure.use(async ({ ctx, next }) => {
  const { user, sessionId } = ctx;
  if (!user || !sessionId)
    throw new TRPCError({
      code: "UNAUTHORIZED",
      message: "You need to log in to access this procedure",
    });
  return await next({
    ctx: {
      user,
      sessionId,
    },
  });
});
```

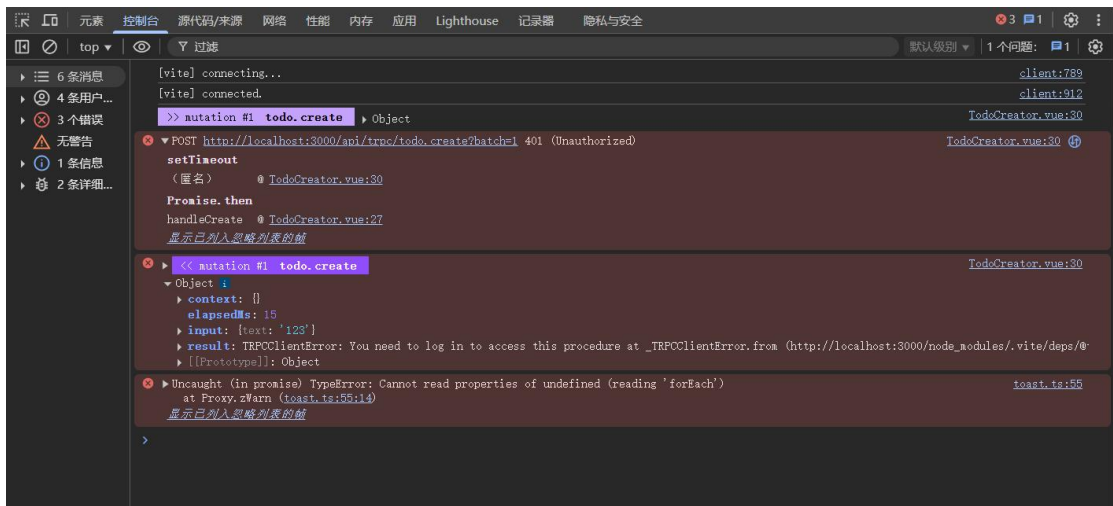


图 4- 9 未授权访问后端接口的错误提示图

4. 3 安全实现

本项目在各个层面都融入了安全考量，以构建一个健壮且受保护的应用。安全实现覆盖了用户身份认证、权限管理以及对多种常见网络攻击的有效防范。

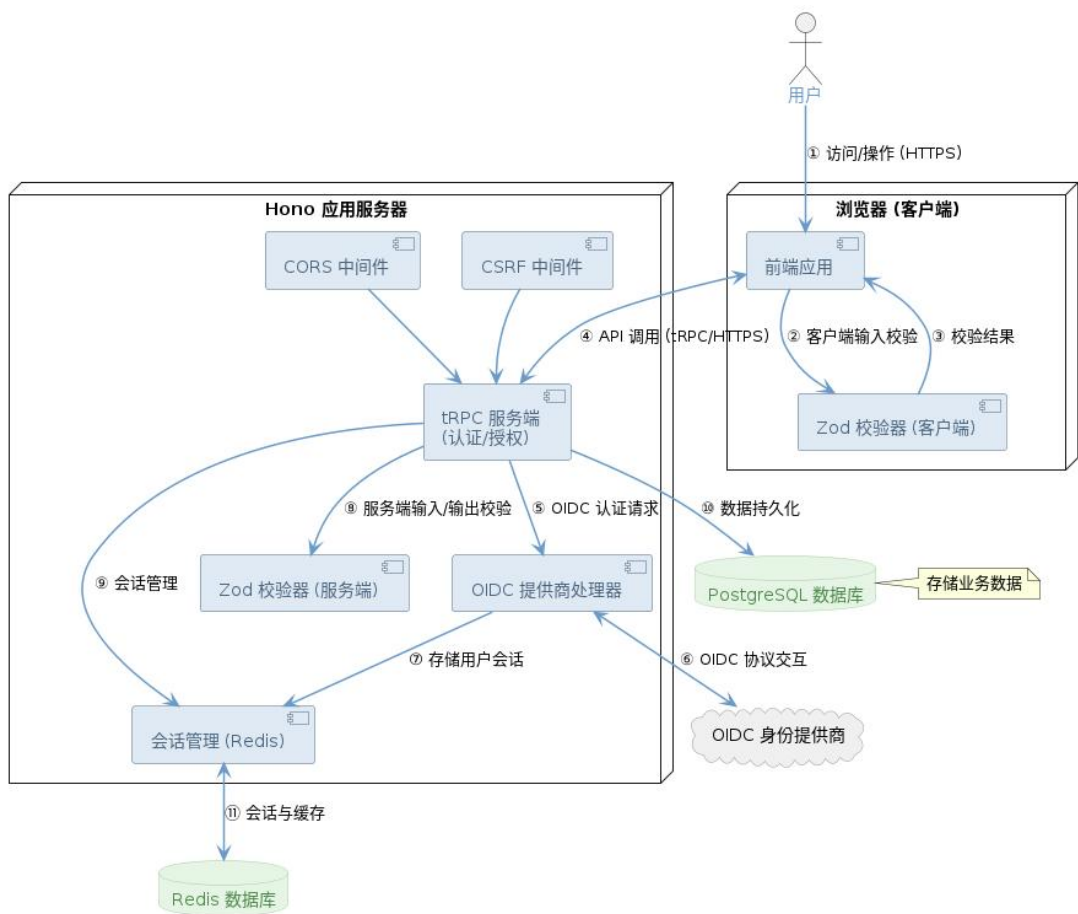


图 4-10 安全整体实现流程图

4.3.1 用户身份认证与权限管理

用户身份认证是系统安全的第一道防线，本项目采用了行业标准化的 OIDC 协议，并配合严格的会话管理和 tRPC 权限控制，确保用户身份的可靠性和操作的授权性。

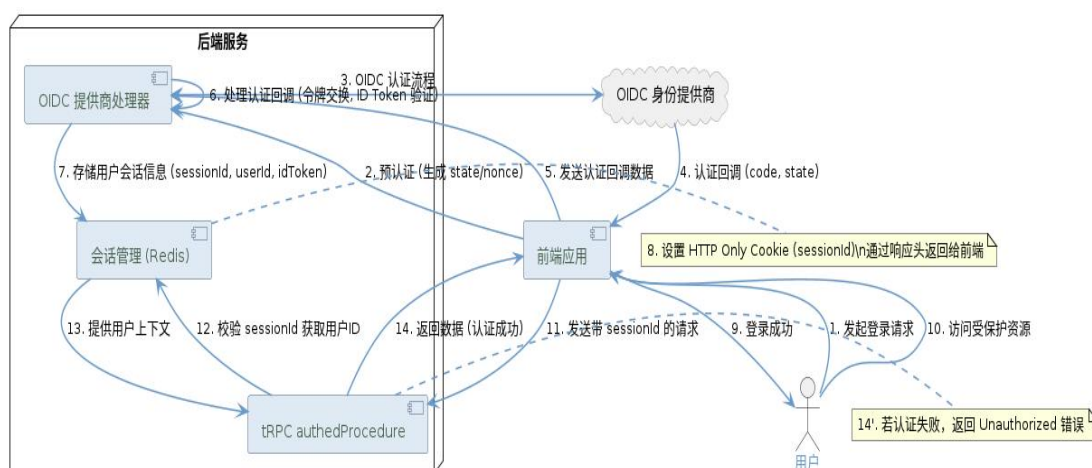


图 4-11 用户认证与权限管理流程图

- **OIDC 认证协议：** 如前所述，通过集成 OIDC 身份提供商 (src/app/server/plugin/oidc-auth-provider/provider.ts)，将用户身份验证的复杂性委托给专业的身份服务，从而利用其成熟的安全机制，包括令牌的颁发、验证和撤销。
- **会话管理：** 用户登录成功后，后端会生成一个 sessionId 并存储在 Redis 中，同时将该 sessionId 以 HTTP Only Cookie 的形式发送给客户端。这种方式可以有效防止跨站脚本 (XSS) 攻击获取会话 Cookie。
- **tRPC 权限控制：** 在后端 tRPC 层，使用 authedProcedure (src/app/server/trpc/server.ts) 对所有需要认证的 API 接口进行封装。在执行这些过程前，系统会检查请求的 sessionId 是否有效以及对应的用户 (ctx.user) 是否存在，确保只有经过认证的用户才能执行特定操作，从而实现了访问控制。例如，所有任务相关的操作 (创建、更新、删除、查询列表) 都受此权限机制保护。

4.3.2 防止常见攻击

项目采取了多项措施来防范常见的 Web 安全攻击，确保数据的完整性、可

用性和机密性。为防止恶意数据注入，所有用户输入在前端和后端都经过严格的验证。前端使用 **Zod** 进行即时校验，提供用户友好的反馈；后端则进行最终且强制的验证，防止绕过客户端验证的攻击。这有效抵御了 SQL 注入、XSS 等基于输入数据的攻击。

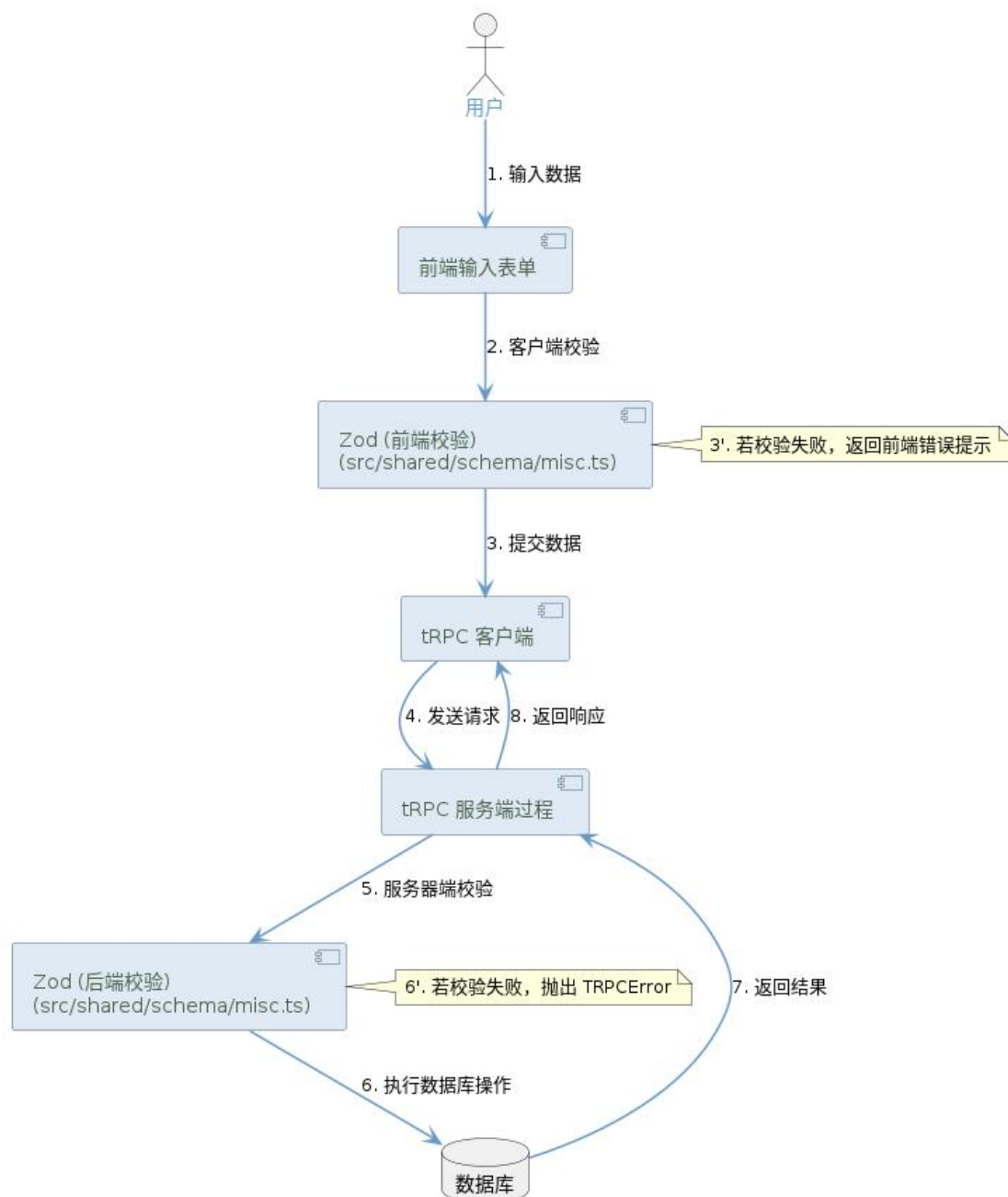


图 4-12 输入验证流程图

示例代码：任务内容输入校验 (`src/app/components/ToDoCreator.vue` 和 `src/shared/schema/misc.ts`)

```

TodoDataSchema.omit({

```

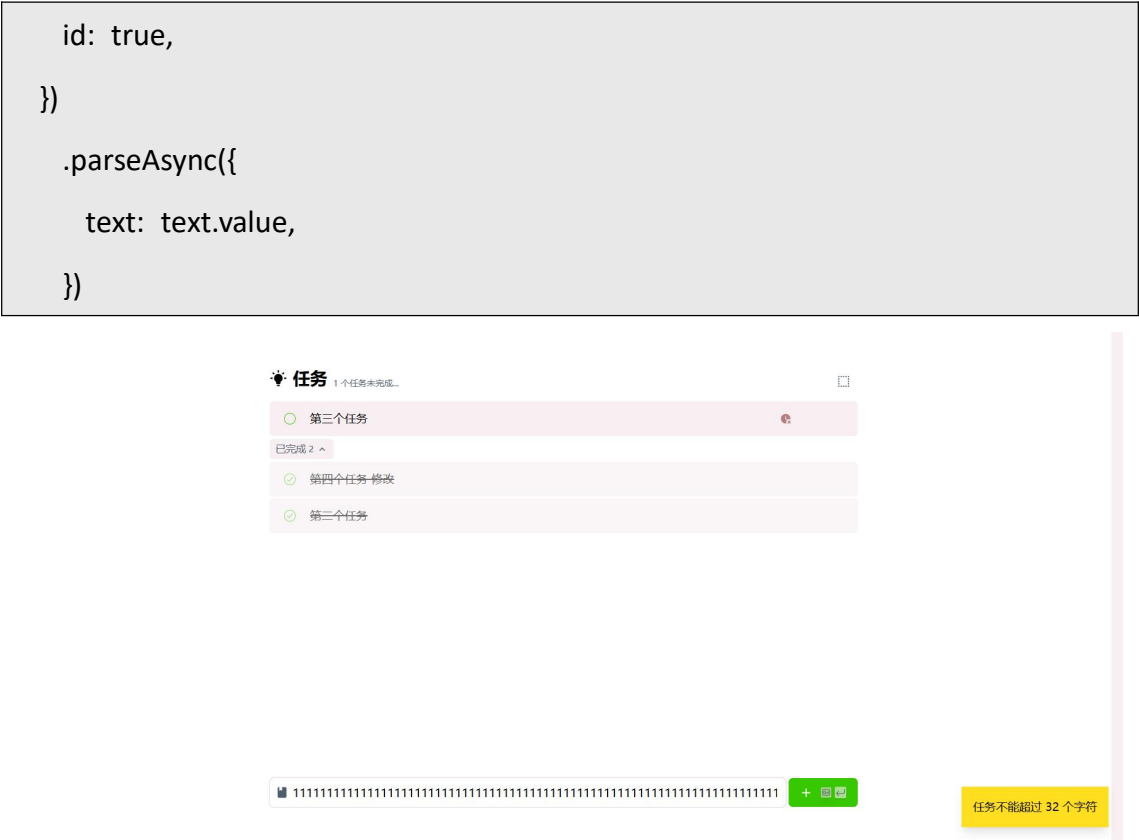


图 4-13 任务创建时的校验提示

五、系统测试

5.1 功能测试

功能测试主要关注应用各个模块的核心业务逻辑是否正确实现，用户交互是否流畅。我们选取了主要的功能点进行测试，力求全面覆盖用户可能的操作路径，以表格形式展示测试详情。

以下功能测试表中，实际结果与预期结果相同时，实际结果的列表项为：

表 5-1 功能测试表

功能项	测试方法	预期结果	实际结果
用 户 登 录 (OIDC)	点击登录按钮，跳转至 OIDC 身份提供商页面，输入凭据后授权并回调。	成功登录，页面重定向至任务列表，并显示用户相关信息。	☑

功能项	测试方法	预期结果	实际结果
用户注销	点击注销/退出登录按钮。	用户会话被清除，页面重定向回登录页或匿名访问主页。	☒
任务创建（有效输入）	在任务输入框中输入有效任务内容（例如：“完成实验报告”），点击“添加”按钮或按下 Ctrl+Enter。	新任务成功添加到任务列表顶部，输入框清空，并有“成功创建任务”提示。	☒
任务创建（无效输入）	在任务输入框中输入空内容、过长内容（超过 32 字符）或只包含空格的内容，点击“添加”按钮。	系统应阻止任务创建，显示相应的错误提示（如“任务必须有内容”或“任务不能超过 32 个字符”），输入框内容保持不变。	☒
任务完成/取消完成	点击任务列表项左侧的复选框，切换任务的完成状态。	任务文本显示/取消删除线，状态更新，后端数据库同步更新，并有成功提示。	☒
任务重要性标记/取消标记	点击任务列表项右侧的星标图标，切换任务的重要性。	任务星标状态更新（实心/空心），任务在列表中根据重要性重新排序，后端数据库同步更新，并有成功提示。	☒
任务文本直接编辑	双击任务列表中的任务文本，进入编辑模式；修改任务内容后，按下回车键或点击输入框外部。	任务文本成功更新，并有成功提示；若输入无效，显示错误提示。后端数据库同步更新。	☒

功能项	测试方法	预期结果	实际结果
任务详情查看 (侧边栏)	点击任何任务列表项。	右侧侧边栏展开，准确显示该任务的完整内容、描述、完成状态、重要性、截止日期等所有详情。	☒
任务描述编辑 (侧边栏)	在任务详情侧边栏中，编辑任务的描述内容。	任务描述成功更新，并有成功提示。后端数据库同步更新。	☒
任务截止日期设置/清除(侧边栏)	在任务详情侧边栏中，通过日期选择器设置或清除任务的截止日期；或使用“今天”、“明天”、“一周内”快捷按钮。	任务的截止日期成功设置或清除，前端显示相应日期，后端数据库同步更新。	☒
任务截止日期提醒(视觉)	设置一个已过期或即将到期的任务截止日期，观察任务列表项。	任务列表项上应有视觉提示(如时钟图标颜色变化)，表示任务已过期或即将到期。	☒
单个任务删除	在任务列表项上悬停，点击右侧出现的垃圾桶图标。	任务从列表中移除，并有“任务已删除”提示，后端数据库同步删除该任务。	☒
任务多选模式切换	点击批量操作工具栏中的“多选”按钮。	进入/退出多选模式，任务列表项左侧出现/消失复选框。	☒
批量选择任务	在多选模式下，点击任务列表项的复选框进行选择/取消	任务被正确选中/取消选中，批量操作工具栏根据选择数	☒

功能项	测试方法	预期结果	实际结果
	选择；或点击批量工具栏中的“全选”/“取消全选”。	量动态变化。	
批量完成/取消完成任务	在多选模式下，选择多个任务，点击批量工具栏中的“勾选”图标，选择“批量完成”或“批量取消完成”。	所有选中任务的状态批量更新，列表显示相应变化，后端数据库同步更新。	☒
批量标记重要/取消重要任务	在多选模式下，选择多个任务，点击批量工具栏中的“星标”图标，选择“批量重要”或“批量取消重要”。	所有选中任务的重要性批量更新，列表排序变化，后端数据库同步更新。	☒
批量删除任务	在多选模式下，选择多个任务，点击批量工具栏中的“垃圾桶”图标。	所有选中任务批量从列表中移除，后端数据库同步删除。	☒
已完成任务显示/隐藏	当有已完成任务时，点击“已完成 x”提示。	已完成任务列表展开/折叠显示。	☒
任务排序（自动）	创建或修改任务的完成状态、重要性、更新时间。	任务列表应自动按照重要性（重要任务在前），然后按更新时间（最新更新在前）进行排序。	☒
主题切换	点击应用顶部的主题切换图标。	应用界面主题在亮色和暗色之间切换，用户选择应被 Cookie 持久化，下次访问时保持一致。	☒

功能项	测试方法	预期结果	实际结果
Toast 通知显示	执行一个会触发通知的操作（例如：创建任务成功）。	屏幕右下角出现相应的 Toast 提示信息（例如：“成功创建任务”），并在指定时间后自动消失。	☒

测试结果截图示例：

以下将展示部分功能测试的关键界面截图，以直观呈现测试结果。在实际测试中，每个功能项都应有对应的截图作为佐证。

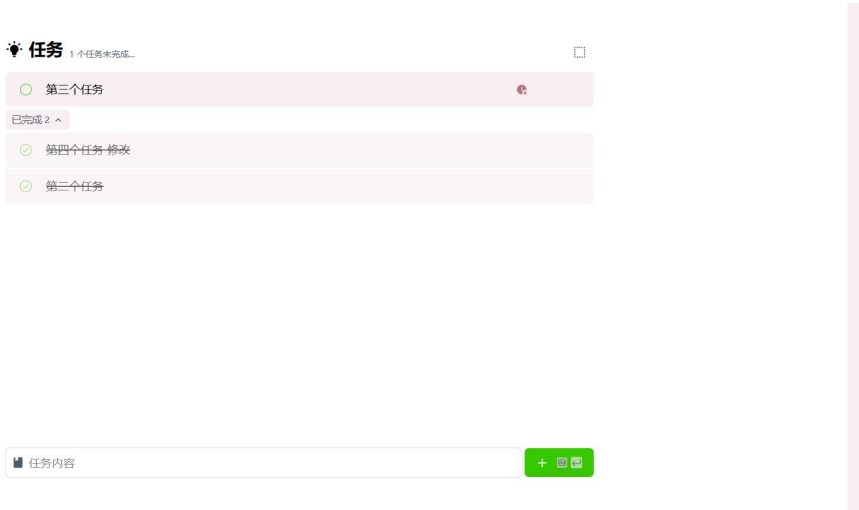


图 5- 1 登录成功后的主界面截图

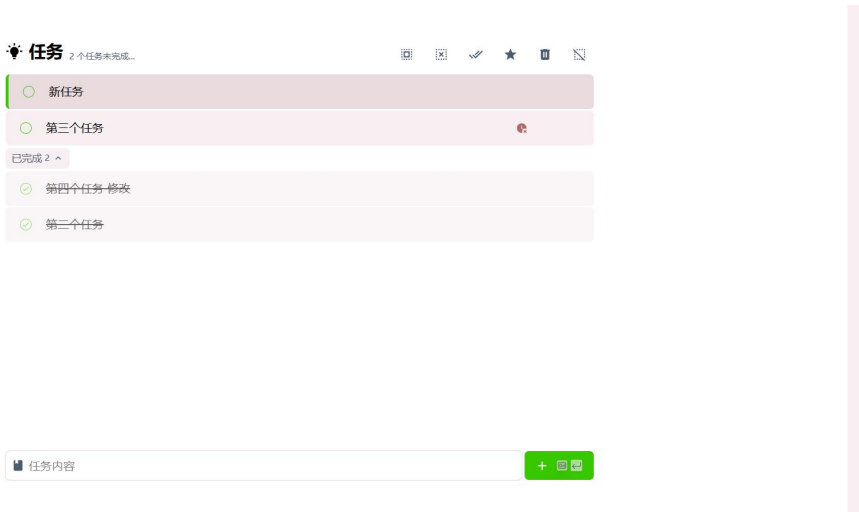


图 5- 2 创建任务后的任务列表截图

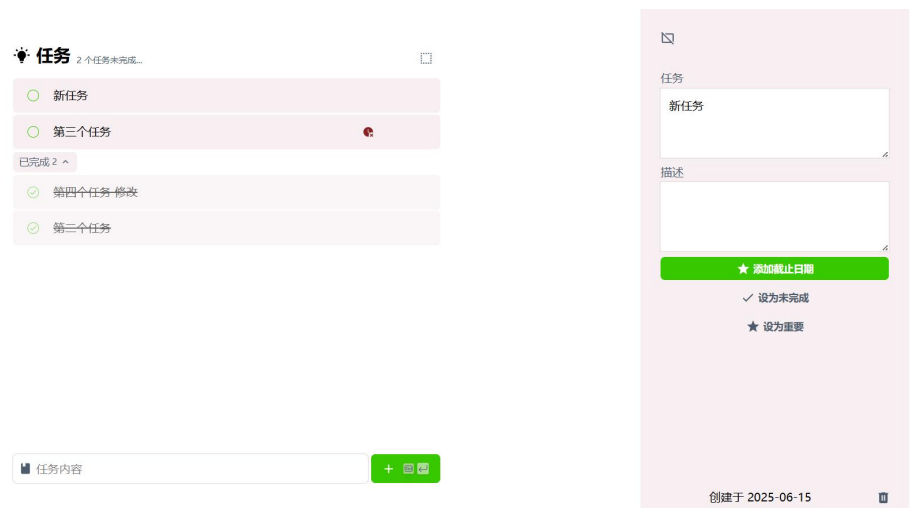


图 5-3 任务详情侧边栏编辑截图

5.2 安全测试

安全测试旨在评估系统在面对恶意攻击时的健壮性。我们将针对常见的安全漏洞进行测试，验证系统是否能够有效防御。

5.2.1 绕过登录测试

- 测试方法：在未登录状态下，尝试直接访问需要认证的后端 API 接口，例如尝试直接通过工具（如 Postman 或浏览器开发者工具的 Console）向 `todo.listOwned` 发送请求，而不携带任何会话 Cookie。观察服务器响应。
- 预期结果：后端 API 应拒绝该请求，并返回 HTTP 401 Unauthorized 状态码或 tRPC 定义的 TRPCError 错误，错误消息明确指出需要登录才能访问此资源。前端在收到此响应后，应能正确地处理并提示用户进行登录。
- 实际结果截图：

任务



图5- 4 绕过登录测试结果截图

5.2.2 SQL 注入漏洞测试

- 测试方法：尝试在所有用户可输入的字段（如任务内容、任务描述）中插入恶意 SQL 片段，例如 `' OR '1'='1` 或 `'; DROP TABLE Todos; --`。提交后，观察后端日志和数据库行为，检查是否存在异常的 SQL 执行或数据被篡改的情况。
- 预期结果：由于使用了 Prisma ORM，所有传入的参数都会被正确地参数化处理，防止 SQL 注入。恶意 SQL 片段应作为普通字符串被存储或直接被 Zod 验证拒绝，不会改变数据库查询逻辑或执行破坏性操作。后端不应出现 SQL 语法错误或未经授权的数据变更。
- 实际结果截图：

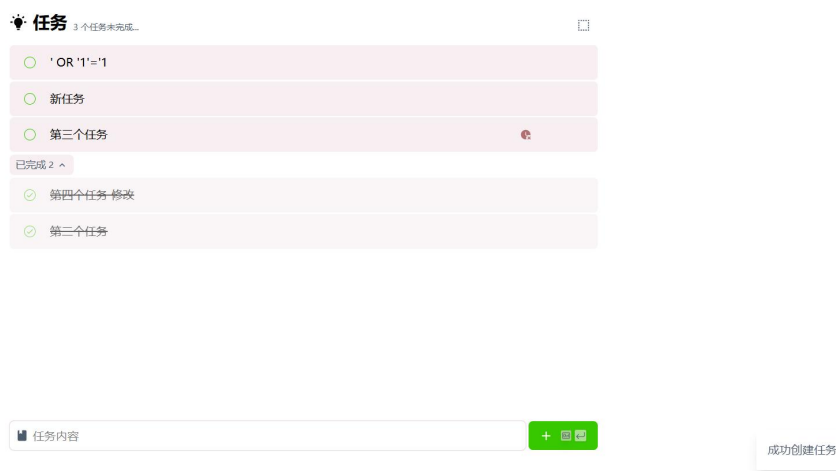


图5- 5 SQL 注入测试结果截图

5.2.3 XSS 漏洞测试

- 测试方法：在所有用户可输入的字段（如任务内容、任务描述）中插入恶意 HTML 或 JavaScript 片段，例如 `<script>alert('XSS Test');</script>`。提交后，观察页面渲染结果，看是否存在弹窗、图片加载失败并触发脚本等异常行为。
- 预期结果：前端在渲染用户输入内容时，应正确地对 HTML 特殊字符进行转义，或者对内容进行净化 (sanitization)，确保恶意脚本不会被浏览器执行。插入的脚本应以纯文本形式显示，或被完全移除，不会造成弹窗或页面行为异常。
- 实际结果截图：

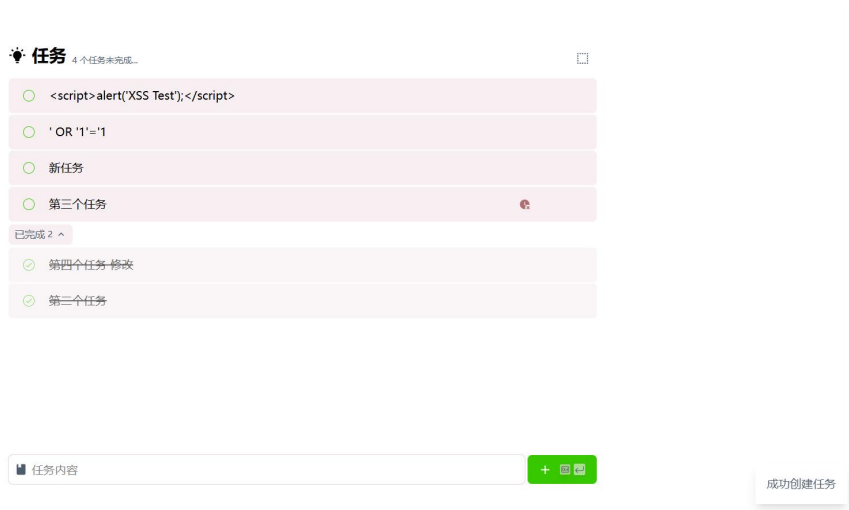


图 5- 6 XSS 漏洞测试结果截图

六、项目总结与反思

这次的 To-Do List 项目，对我们来说确实是一次挺有意思的经历。虽然整个项目从技术栈的选择到架构的设计，基本上都在我们日常学习和实践的范围内，没有遇到什么“史诗级”的难题，但深入去把一个完整的小型应用从头到尾搭起来，并且考虑到用户体验和安全方面，还是有很多值得回味和总结的地方。

6.1 本次实习/项目中学习了哪些新技术？

在这次项目中，我们团队主要接触并巩固了几项新技术，虽然有些在日常实践中都有所涉猎，但又一次在新的实际项目中应用并解决具体问题，感受还是不一样的：

- **Vue 3 & Composition API:** 我们深入使用了 Vue 3 的 Composition API，相比之前的 Options API，它在逻辑复用和代码组织上确实更灵活，让大型组件的代码可读性提升了不少。特别是在处理任务的各种状态（完成、重要性、编辑等）时，用 `ref`、`computed` 和 `watch` 搭配 `Pinia` 管理全局状态，感觉整个前端逻辑清晰了很多。
- **Vike:** 这个 SSR 框架给了我们很深的印象。它让服务器端渲染的配置变得非常简单，只需要遵循文件系统路由的约定，就能实现开箱即用的 SSR，这对于首屏加载性能和 SEO 都很有帮助。Vike 提供的各种生命周期钩子，例如数据预加载和 `Pinia` 状态水合，让我们对同构应用有了更直观的理解。
- **tRPC:** tRPC 是这次项目的一大亮点。它真的解决了前后端接口定义和维护的“痛点”，特别是类型安全。以前写 RESTful API，前端调用时经常要对照后端文档，手动定义类型，一不小心就出错了。tRPC 让我们在前端直接调用后端函数，而且所有类型都是自动推导的，开发体验非常顺畅，大大减少了接口联调和调试的时间。
- **Hono 框架:** 后端选择了 Hono，它作为一个轻量级的 Web 框架，在处理 HTTP 请求和中间件方面表现出色。虽然它的生态不如 Express 那么庞大，但在我们的项目中，其精简和高性能的特点得到了很好的体现，配合 tRPC 更是如虎添翼。
- **Prisma ORM:** 使用 Prisma 让我们对数据库操作有了全新的认识。它的类型安全和自动迁移功能，让数据库模型的管理变得异常轻松。再也不用手写复杂的 SQL 语句，而且在 TypeScript 环境下，数据库字段的类型检查也为我们减少了许多潜在的运行时错误。特别是事务管理，在批量操作任务时，保证了数据的一致性，用起来非常方便。
- **Redis (会话管理):** Redis 在项目中主要用于 OIDC 认证流程中的临时状态存储和用户会话管理。它的高速读写能力，确实为认证流程的效率提供了保障。虽然只是简单的键值存储，但对我们理解分布式缓存和会话机制有了更深的认识。
- **Zod (数据校验):** Zod 的引入让数据校验变得异常优雅。无论是前端的用户输入还是后端接收到的请求体，都能通过 `Zod Schema` 进行统一的、强大的校

验。它提供的丰富校验规则和友好的错误提示，不仅提升了用户体验，也为后端的数据处理提供了第一道安全防线。

- **OIDC 协议与身份认证:** 这次项目有机会接触了标准的 **OpenID Connect (OIDC)** 协议，这让我们对现代身份认证有了更深入的了解。理解授权码流、ID Token 和 Access Token 的作用，以及如何与身份提供商交互，这些都让我们对构建安全可靠的用户认证系统有了更清晰的认知。虽然实现细节比较复杂，但通过使用现有的库和插件，我们还是顺利地完成了集成。

6.2 哪些部分最具挑战？如何克服？对于项目开发流程的理解有何提升？

说实话，在这次 **To-Do List** 项目的开发过程中，对我们团队来说，并没有遇到什么特别“难啃”的技术挑战。这主要是因为我们在做这个项目之前，就已经对前后端分离、Vue、Node.js、数据库操作以及基本的 Web 安全概念有了比较扎实的基础和实践经验。

- “挑战”的内涵：如果非要说有挑战，那更多的是在如何将这些我们已经熟悉的“点”更好地“连成线”，并追求“工程化”和“代码美学”上的提升。比如，如何让 **tRPC** 的类型推导发挥到极致，如何更优雅地组织 **Pinia Store** 的模块，或者在不引入过多复杂性的前提下，实现一套清晰的错误处理机制。这些更像是对现有知识的深度挖掘和精细化打磨，而非从零开始攻克未知领域。
- 克服与深化：我们克服这些“挑战”的方式，主要还是通过细致的代码设计、模块间的职责划分，以及严格的代码审查和测试。例如，在 **tRPC** 的错误处理上，我们约定了统一的错误码和错误信息格式，前端通过 **useToastStore** 统一展示，确保了用户体验的一致性。在组件设计上，我们尽可能地遵循单一职责原则，让每个组件只做一件事，从而提高复用性和可维护性。
- 对项目开发流程的理解提升：尽管对开发流程已较熟悉，但这次项目让我们对以下几点有了更深的体会：
 - ✓ 类型安全的重要性：**tRPC** 和 **Zod** 在整个开发流程中，极大地提升了开发效率和代码质量。这种从数据模型到接口、再到前端组件的端到

端类型安全，让我们在开发过程中能更早地发现问题，而不是等到运行时才暴露，这简直是开发者的福音。

- ✓ 工程化思维的实践：从前端的组件化、状态管理、SSR 集成，到后端的路由设计、ORM 使用、Redis 缓存，再到整个项目的目录结构和环境配置，我们更加系统地实践了工程化思维。理解不同模块如何协作、如何通过配置而非硬编码来提升灵活性，这些都是在真实项目中才能深刻体会到的。
- ✓ 完整闭环的体验：这次项目让我们完整地走过了从需求分析、技术选型、架构设计、前端开发、后端开发、数据库集成、安全加固到最终测试的整个生命周期。这种从“纸上谈兵”到“落地实现”的闭环体验，让我们对每一个环节的投入和产出有了更全面的认知。
- ✓ 工具链的整合与效率：我们对所选用的技术栈及其工具链的整合运用更加熟练。Vite 的开发服务器、Hono 的中间件、Prisma 的迁移工具等，这些工具在整个开发流程中发挥了巨大的提效作用。理解如何高效利用这些工具，是提升开发效率的关键。

总的来说，这次项目与其说是技术上的突破，不如说是对我们现有技术栈的一次深度整合和精细化演练，让我们对如何构建一个高内聚、低耦合、安全且高效的 Web 应用有了更系统、更深刻的理解。

6.3 若有机会优化系统，将重点改进哪些部分？

如果未来有机会对当前 To-Do List 系统进行进一步的优化和迭代，我们团队会着重考虑以下几个方面，旨在提升用户体验、系统性能和可扩展性：

- 任务分类与标签管理：目前任务只有基本的完成状态和重要性。如果能增加任务分类（如工作、生活、学习）或者自定义标签功能，用户就能更灵活地组织和筛选任务，极大提升任务管理的效率和用户体验。这可能需要增加新的数据库表和前端界面来支持分类/标签的创建、编辑和分配。
- 高级搜索与筛选功能：当前任务列表的筛选和搜索功能相对基础。未来可以实现更强大的搜索能力（支持关键词模糊搜索、多条件组合筛选，如按截止

日期范围、按分类/标签、按完成状态、按创建者等），让用户能快速定位到所需任务。

- 用户协作与共享功能：作为一个 To-Do List 应用，如果能支持任务的共享和协作，比如将一个任务分配给团队成员，或者创建一个共享的任务列表供多人共同管理，将会极大地拓展应用的使用场景和价值。这会涉及复杂的权限管理、实时通信（WebSocket）以及通知机制。
- 用户自定义视图与排序：虽然现在任务有默认排序规则（重要性优先，然后按更新时间），但如果能允许用户自定义任务列表的排序方式（如按字母顺序、按创建时间、按截止日期），并能保存这些自定义视图，将能更好地满足不同用户的使用习惯。
- 离线支持与数据同步：为了提升在网络不稳定环境下的用户体验，可以考虑引入离线存储（如 IndexedDB），实现用户在无网络连接时也能查看和管理任务。待网络恢复后，再将本地修改同步到服务器，这会增加客户端数据同步的复杂性。
- 集成第三方服务：例如，集成日历服务（如 Google Calendar、Outlook Calendar）将任务的截止日期同步到用户的个人日历中，或者集成通知服务（如邮件、短信）提供更丰富的任务提醒方式。
- 性能优化（缓存策略深化）：虽然目前使用了 Redis 进行会话管理，但在数据读取方面还可以进一步优化缓存策略。例如，对热点任务列表数据进行更细粒度的缓存，或者引入 CDN 加速静态资源的加载。
- 国际化 (i18n) 支持：当前应用主要是中文界面，如果未来有推广到国际用户的需求，增加国际化支持将是必要的，让应用可以支持多语言。

这些优化方向将使 To-Do List 应用更加完善和强大，满足更广泛的用户需求，并提升整体的用户满意度。

七、参考文献

在本次 To-Do List 项目的开发过程中，我们主要依赖了各技术栈的官方文档、教程以及在线知识库。这些资料为我们提供了最权威、最详尽的技术指导和最佳实践，确保了项目实现的正确性和高效性。虽然没有直接引用学术论文或期刊文章，但对这些官方手册的深入学习和理解，是我们项目顺利进行并完成的关键。

以下是我们本次项目过程中主要查阅和参考的官方资料：

- [1] Vue.js. (n.d.). Vue.js Documentation.
- [2] Pinia. (n.d.). Pinia Documentation.
- [3] Vike. (n.d.). Vike Documentation.
- [4] tRPC. (n.d.). tRPC Documentation.
- [5] Hono. (n.d.). Hono Documentation.
- [6] Prisma. (n.d.). Prisma Documentation.
- [7] Redis. (n.d.). Redis Documentation.
- [8] Zod. (n.d.). Zod Documentation.
- [9] OpenID Foundation. (n.d.). OpenID Connect Core 1.0.
- [10] MDN Web Docs. (n.d.).
- [11] UnoCSS. (n.d.). UnoCSS Documentation.
- [12] Vite. (n.d.). Vite Documentation.